



Department of Mathematics and Computer Science

Master's thesis

Novel neighborhood-based feature encodings for molecules

Nils Neumann

March 31st, 2021

Dr. Georges Hattab
Prof. Dr. Dominik Heider

Erklärung zur Abschlussarbeit

Hiermit versichere ich, Nils Neumann, die vorliegende Master-Thesis / Bachelor-Thesis ohne Hilfe Dritter und nur mit den angegebenen Quellen und Hilfsmitteln angefertigt zu haben. Alle Stellen, die Quellen entnommen wurden, sind als solche kenntlich gemacht worden. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Mir ist bekannt, dass im Falle eines Plagiats ein Täuschungsversuch vorliegt, der dazu führt, dass die Arbeit mit unter 5 Punkten bewertet und damit ein Prüfungsversuch verbraucht wird. Abschlussarbeiten dürfen nur einmal wiederholt werden.

Bei der abgegebenen Thesis stimmen die schriftliche und die zur Archivierung eingereichte elektronische Fassung überein.

Thesis Statement

I herewith formally declare that I, Nils Neumann, have written the submitted thesis independently. I did not use any outside support except for the quoted literature and other sources mentioned in the paper. I clearly marked and separately listed all of the literature and all of the other sources which I employed when producing this academic work, either literally or in content. This thesis has not been handed in or published before in the same or similar form.

I am aware, that in case of an attempt at deception based on plagiarism, the thesis would be graded under 5 points and counted as one failed examination attempt. The thesis may only be repeated once.

In the submitted thesis the written copies and the electronic version for archiving are identical in content.

Ort/Place, Datum/Date

Unterschrift des Autors/Signature of author

Abstract

With increasing computational capabilities and advanced algorithms, computer-aided approaches have become a popular tool to assist biological, chemical, and medical research. For instance, the discovery of novel therapeutics can be accelerated by virtual screening or the prediction of biological properties with machine learning. However, to process biological or chemical data with computer algorithms, the data first needs to be encoded.

In this work, a novel, parametric molecular encoding for organic chemistry is developed. The proposed encoding generates a descriptor for the input molecule by recording and transforming the neighborhoods surrounding its carbon skeleton. The neighborhoods are defined hierarchically based on their proximity to the carbon chain. This way, different depths of hierarchy can be recorded to best suit the input compound. Subsequently, the encoding is evaluated on eight data sets containing small organic compounds in a binary classification task.

For evaluation, a comprehensive machine learning workflow is implemented. Three feature selection approaches (PCA, variable importance, and RFE) are combined with two classification models (random forest and SVM).

Very promising classification results were achieved with the proposed encoding. For the evaluated data sets, ROC AUC values of up to 0.89 were reached when combining a random forest classifier with recursive feature elimination.

The results support the concept of generating machine-readable molecular representations by recording and transforming the neighborhoods around the carbon chain.

Contents

Abstract	5
1 Introduction	9
1.1 Background and Motivation	9
1.2 Goals	10
1.3 Structure	11
2 Related Works	13
2.1 Molecular Fingerprints	13
2.2 Applications	16
2.3 Similarity Measures for Molecular Fingerprints	17
2.4 Machine Learning with Molecular Fingerprints	18
2.5 Protein and Peptide Classification	19
3 Materials	21
3.1 Data	21
3.1.1 Amino Acids	22
3.1.2 Data Sets	22
3.1.3 Data Formats	23
3.1.4 Data Structure	24
3.2 Software	24
3.2.1 Python	25
3.2.2 Python Packages	25
3.2.3 R	26
3.2.4 R Packages	27

4	Methods	29
4.1	Data Pre-Processing	29
4.2	Neighborhood Encoding	31
4.2.1	Molecule Graph	31
4.2.2	Molecular Neighborhoods	32
4.2.3	Feature Transformation	36
4.2.4	Padding the Encoding	40
4.2.5	Image Generation	42
4.3	Evaluation	44
4.4	Dimensionality Reduction	44
4.4.1	Feature Pre-Processing	45
4.4.2	Feature Selection	46
4.4.3	Feature Selection and Validation	49
4.5	Machine Learning	50
4.5.1	Validation	51
4.5.2	Metrics for Evaluation	52
4.5.3	Total Number of Models	53
5	Results	55
5.1	Dimensionality Reduction	55
5.2	Classification Results	56
5.3	Image Representations	59
6	Discussion	63
7	Conclusion	67
	Terms and Acronyms	69
	References	73

1. Introduction

The following sections provide the background information necessary to understand the terminology used throughout the thesis. Furthermore, applications of molecular encodings, and the motivation behind the chosen feature representation is given. Lastly, the structure of the work is outlined.

1.1 Background and Motivation

Computer-aided approaches are a popular tool to support biological, chemical and medical research. Their applications include the discovery of novel therapeutics [1], analyzing drug-target interactions [2], or the prediction of biological or chemical properties using machine learning [3]. All approaches share one common step. Before applying algorithms to biological or chemical data, the data has to be transformed into a machine-readable format. This process is known as encoding. Based on the type of data and the specific task, numerous encodings exist that make use of different characteristics of the underlying compounds [4].

For molecules specifically, the encoding process is referred to as *molecular fingerprinting* [5]. The different types of molecular fingerprints as well as their applications and limitations are discussed in detail in Chapter 2: Related Works. The goal of this work is to conceptualize, implement and evaluate a new type of molecular encoding for organic chemistry. In contrast to other existing encodings, the structure of the encoded compound on an atomic level is taken into account. Additionally, the proposed encoding shifts the focus away from the entire molecule and instead concentrates specifically on the carbon chain and its surrounding neighborhoods.

As the "element in the periodic table that provides the basis for life" [6, p. 1], the carbon element acts as the focal point of the proposed encoding. It is investigated whether recording the neighborhoods surrounding the carbon chain of organic compounds can produce a suitable molecular representation for machine learning. For this purpose, the proposed encoding is applied to data sets consisting of small or-

ganic compounds, like peptides. The data sets are compiled from different research papers. Subsequently, the encoded data sets are used to predict biological properties using a comprehensive machine learning workflow.

In addition to developing the encoding itself, this work also focuses on examining the generated feature vectors from a statistics and data science standpoint. Specifically, statistical operations and feature selection methods are applied to the encoded data sets to gain insights about the molecules’ structures. During this process, the parameter settings of the proposed encoding are tweaked in an effort to achieve optimal performance.

1.2 Goals

The goal of this work is to develop a novel encoding for organic compounds based on the neighborhoods surrounding the carbon chain. Multiple levels of neighborhood are defined hierarchically based on their proximity to the carbon chain. Subsequently, these are referred to as *hierarchies*. By incorporating different numbers of hierarchies in the encoding, compounds of varying sizes and containing different substructures can be represented appropriately.

Four main requirements are defined for the proposed encoding.

First, in order to be applicable to large data sets, the encoding needs to be computationally efficient.

Second, the feature vectors generated by the proposed encoding need to be compatible with all tasks typically supported by other molecular fingerprints. For instance, this includes database querying [7] and virtual screening [5]. Chapter 2 contains a comprehensive introduction into the applications of molecular fingerprints.

Third, in order to handle the vast variety of compounds belonging to organic chemistry, parameter settings to control the encoding need to be made available to the user. This way, the encoding can be tuned to best suit the available data.

Lastly, the encoding needs to be accessible even to users without explicit domain knowledge of computer science or algorithm design. Research areas supported by molecular fingerprints range from biology and chemistry to medical research. Therefore, it is important that the methodology as well as the generated feature vectors can be represented in an intuitive manner. Specifically, the encoding algorithm needs to offer visual representations to complement the mathematical feature vectors. In addition to increasing accessibility, visual representations can enable additional operations in image space [8].

While satisfying those requirements, the encoding needs to be compatible with established standards of bioinformatics and cheminformatics. Most importantly, this includes the data format used to input molecular data into the encoding algorithm. Computer applications relating to bioinformatics and cheminformatics rely on common data and file formats. The proposed encoding needs to offer compatibility to established standards of data representation. The relevant file formats and data structures are introduced in detail in Section 3.1.3: Data Formats. Furthermore, to compare different compounds, it is necessary to quantify the similarity between their molecular fingerprints. Most commonly, the Tanimoto coefficient is used as a similarity measure for this purpose [5]. Chapter 2: Related Works offers an overview over the important similarity measures. Therefore, the proposed encoding needs to output the generated feature vectors in a format compatible with the most common similarity measures. Specifically, this means the output feature vectors need to be either binary or numeric. Lastly, the goal of the image representations generated in addition to the numeric encoding is to increase accessibility. Because of this, the types of elements encoded in the feature vectors need to be immediately obvious from looking at the corresponding image representations.

In order to evaluate the proposed encoding in a comprehensive manner, the evaluation task needs to simulate a real-world problem while featuring a variety of data sets. Hence, the problem to solve for evaluation will consist of predicting the presence or absence of biological properties in different data sets using machine learning. The variety between the data sets needs to be ensured by including data with varying numbers of observations and class distributions. The data sets along with their predicted properties are introduced in detail in Section 3.1: Data. Additionally, bias within the machine learning workflow based on feature selection approach or model choice needs to be minimized.

1.3 Structure

To begin with, the Related Works chapter will give an overview over past and present research related to the applications and challenges of encoding molecules.

Next, the data and software used as part of this work are presented.

Afterwards, Chapter 4 introduces the methodology for developing and evaluating the proposed encoding. The methodology consists of two major parts. First, the encoding algorithm is implemented. This includes handling the input data, generating intermediate data representations, traversing the molecule and recording the rele-

vant neighborhoods according to specified parameters, transforming the recorded features to their final output format, and generating the corresponding image representations. The second part of the methodology introduces the machine learning pipeline implemented for evaluating the encoding. As part of the evaluation pipeline, the topics dimensionality reduction, model training, validation strategies, and evaluation metrics are covered.

The feature vectors resulting from encoding the data sets, the performance metrics calculated as part of the machine learning workflow, and the generated images representations are presented and discussed in the concluding chapters.

2. Related Works

The following chapter introduces works relating to the topic of molecular encoding, subsequently also referred to as molecular fingerprinting. First, common terminology is defined and current techniques for molecular fingerprinting are explored. In addition, select molecular fingerprints are introduced in detail and their strengths and weaknesses are discussed. Next, common applications for molecular fingerprints are highlighted, including virtual screening. Lastly, studies covering the application of machine learning models on data encoded by molecular fingerprints are discussed, both in a general manner and with specific focus on peptide classification.

2.1 Molecular Fingerprints

Molecular fingerprints are used to encode molecules into a machine-readable data format. This allows researchers to apply mathematical transformations to the molecule, enabling statistical analysis and machine learning techniques to discover new insights about the underlying compound. Working with fingerprint representations also decreases computational expenses and enables rapid comparison of different molecules [9]. A variety of fingerprints exist which make use of different characteristics of the molecule and result in different data structures. Most commonly, binary fingerprints encode the molecule into a sequence of binary digits - so called bits [10]. However, any numeric representation of a molecule can be considered a fingerprint [5]. These representations can then be used to compare molecules based on different distance measures [11], to search databases [7], or to train machine learning models to predict chemical or biological properties [3].

Molecular fingerprints can be categorized into different groups, with the main approaches being "substructure keys-based fingerprints, topological or path-based fingerprints, and circular fingerprints" [7, p. 59].

Substructure key-based

Substructure key-based fingerprints set the bits of the resulting feature vector depending on whether or not specific substructures are present within the molecule. The length of the feature vector corresponds to the number of substructure keys. Since key-based fingerprints rely on previously defined substructure keys, these types of fingerprints tend to only perform well for molecules containing the structural keys. Otherwise, their features cannot be represented [7]. To combat this shortcoming, "several programs allow for customized keysets" [5], enabling the user to update the available keys to cover the relevant substructures. Popular key-based fingerprints include the Molecular Access System (MACCS) fingerprint [12], developed by MDL information systems, as well as the PubChem fingerprint [13], used by the PubChem website for similarity searching. In a paper published in 2018, Wang *et. al.* [14] employed the use of the MACCS and PubChem molecular fingerprints for ADMET prediction. AUC values of 0.895 and 0.879, respectively, were achieved, highlighting the versatility and accuracy of substructure key-based fingerprints.

Topological

In contrast, topological fingerprints are not reliant on predefined substructures. Instead, they analyze a molecule by tracing a path up to a predefined number of bonds. In doing so, topological fingerprints hash the discovered features using a hashing function into a bit sequence. This results in two distinct characteristics. Firstly, the bits in the resulting bit sequence (i.e. the fingerprint’s features) cannot be traced back to their original fragment in the molecule, meaning topological fingerprints inherit an additional layer of abstraction from the hashing function. Secondly, since the number of bits in the resulting bit sequence is finite, it can occur that multiple different features are hashed to the same bit. This is called *bit collision* and results in a loss of information. Bit collisions can be circumvented up to a certain point by making more bits available to the fingerprint, however this results in slower computational performance [7, 5]. An example of a path-based topological fingerprint can be found in FP2, used by the popular chemistry software toolbox OpenBabel [15].

Circular

Circular fingerprints encode molecular information by recording the environment of each atom up to a predefined radius. These types of fingerprints are not applicable to the task of substructure querying, but instead are commonly used for full structure similarity searching [7]. In circular fingerprinting, only environments within the predefined radius are encoded, meaning the descriptor generated for each atom

is local instead of global. Additionally, the encoded environments generally do not contain information about the chemical bonds between atoms [16].

As the "*de facto* standard of circular fingerprints" [5], the Extended-Connectivity Fingerprints (ECFPs) [17] are used in a number of studies. For instance, in a study published in 2007, researchers used ECFPs in combination with a k-Nearest Neighbor approach to predict cytochrome enzyme inhibition for avoiding drug-drug interactions [18].

Additionally, the Multilevel Neighborhoods of Atoms (MNA) [19] circular fingerprint shares conceptual similarities to the molecular encoding developed as part of this work. The MNA fingerprint generates a complete descriptor of the molecule. It does so iteratively by appending each atom along with a lexicographically ordered list of its neighbors to the descriptor until a predefined distance from the origin is reached. In contrast to the encoding introduced as part of this work, the MNA fingerprint abstracts the chemical elements into 32 groups, each represented by a unique symbol. To evaluate their fingerprint, Filimonov *et. al.* [19] predicted the boiling points of 139 hydrocarbons using the MNA fingerprint and compared them to the observed boiling points of the respective compounds. With this method, the authors were able to achieve a "highly significant correlation between the observed and predicted boiling points" [19, p. 669].

Other Fingerprints

There also exists a number of other fingerprinting techniques not strictly belonging to the aforementioned categories. These include **hybrid fingerprints** that combine characteristics from other fingerprints, like the Unity 2D fingerprint developed by Tripos Inc. (now Certara) [20]. The Unity 2D fingerprint generates a 988-bit sequence for the given molecule by combining structural keys and a path-based approach. **Pharmacophore fingerprints**, on the other hand, encode information about a molecule relating to its "features [...] necessary to ensure the optimal supramolecular interactions with a specific biological target" [21, p. 1140]. In addition to the common application of finding similarities between different molecules, pharmacophore fingerprints can also be used to analyze factors contributing to the molecule's biological function [22].

2.2 Applications

Since molecular fingerprints act as a supplementary step to many types of analyses, their applications are numerous. Molecular fingerprints are used across many domains like biology, chemistry, and medical research.

One of their main uses is in virtual screening. In virtual screening, "novel bioactive molecules are sought based on their similarities to [...] reference compounds" [5]. Finding new compounds similar to existing ones is an important task in drug discovery, since it allows researchers to pre-select a much smaller number of potentially interesting candidates for experimentation instead of having to test "hundreds or thousands of compounds" [7, p. 58]. This type of virtual screening via similarity searching is made possible by the *Similar Property Principle*, stating that molecules with similar structure also display similar properties [23]. Virtual screening via similarity searching is carried out by first calculating similarity measures for all compounds in a database to the compound with desirable properties, the so-called reference compound. Afterwards, they are ranked by decreasing similarity and manual screening is carried out on the compounds with the highest similarity rankings [24]. This method can be extended to account for more than one bioactive reference compound. In this scenario, research has shown that fusing similarity scores from multiple individual searches generally yields the best results [25]. In addition to similarity searching, quantitative structure-activity relationship (QSAR) models can be used for virtual screening. QSAR "is a computational modeling method for revealing relationships between structural properties of chemical compounds and biological activities" [26, p. 1]. With QSAR, researchers attempt to build regression or classification models to represent correlations between the desired biological property and the available chemical information [27].

Virtual screening is predominantly used in drug discovery, for instance to find treatments for Alzheimer's disease [28] or HIV [29]. Additionally, in a study published in 2008, QSAR was used for modeling antimicrobial peptides as a novel approach to drug development [30].

2.3 Similarity Measures for Molecular Fingerprints

Comparison of molecular fingerprints for similarity searching can be achieved using different similarity measures depending on the type of fingerprint used. Commonly, molecular fingerprints generate feature vectors containing bit sequences to describe the underlying compound. Therefore, any distance measure suitable for binary data can be used to compare compounds encoded by these types of molecular fingerprints. In order to convert distance to similarity and vice-versa, the following equation can be used [31]:

$$similarity = \frac{1}{1 + distance}$$

Note that this assumes the distance measure to be non-negative. An overview of different appropriate distance measures for dichotomous variables can be found in Table 2.1. While the Tanimoto coefficient "is the most frequently applied similarity measure for bit strings" [5], Rácz *et. al.* [32] have concluded that there are other similarity metrics more appropriate for protein data sets than the Tanimoto coefficient. Moreover, it has been shown that statistical effects such as combinatorial preferences can influence similarity calculations performed with the Tanimoto coefficient [33]. Nevertheless, Bajusz *et. al.* [11] have identified the Tanimoto, Dice, Cosine, and Soergel similarities as the best and equivalent similarity measures.

Table 2.1: Distance and similarity measures for binary data [11].

Distance measure	Formula
Tanimoto coefficient	$\frac{c}{a+b-c}$
Euclidean distance	$\sqrt{a+b-2c}$
Manhattan distance	$a+b-2c$
Dice coefficient	$\frac{2c}{a+b}$
Cosine similarity	$\frac{c}{\sqrt{ab}}$
Soergel distance	$\frac{a+b-2c}{a+b-c}$

Given two bit sequences A and B ,

a equals the number of bits set to 1 in A

b equals the number of bits set to 1 in B

c equals the number of bits set to 1 in both A and B

In addition to the purely mathematical approach of calculating similarity measures for molecular fingerprints, work has been done to explain the underlying factors of a molecule that determine its similarity value or machine learning model prediction. In a paper published in 2013, Riniker *et. al.* [9] introduced *similarity maps* for visualizing fingerprint similarities as well as model predictions. These similarity maps are built by first calculating a weight value for each atom in the molecule, and subsequently overlaying a color map over the molecule graph with the color values corresponding to the normalized weight values calculated for the atoms.

2.4 Machine Learning with Molecular Fingerprints

In order to use machine learning methods for regression or classification tasks, the data first has to be transformed into a suitable data format. Molecular fingerprints can be used to encode molecular data into a machine-readable format. This allows for the application of machine learning techniques to solve tasks from biology, chemistry or medicine such as the identification of drug-target interactions (DTIs) [2], or virtual screening [34]. Additionally, feature selection or feature ranking approaches can be used to identify the parts of the underlying compound with the strongest effect on the outcome of a given task. In a study published in 2018, feature selection for virtual screening was performed using deep learning methods. The authors found that using a deep belief network (DBN) for feature selection can improve the performance of ligand-based virtual screening [34]. Additionally, Bender *et. al.* [35] were able to "outperform[] all current retrieval methods [...]" [35, p. 170] in database searching by combining a count vector-based fingerprint with information gain feature selection and a Naïve Bayes classifier.

Machine learning approaches can also be used to predict DTIs. Saad *et. al.* [36] found that using a combination of molecular fingerprints and drug side effects as features, DTIs can be identified with an accuracy of up to 94.69%.

The aforementioned studies highlight the versatility of molecular fingerprints as a data representation for machine learning tasks. Research into additional applications of molecular fingerprints for machine learning and state of the art deep learning approaches is ongoing and further developments can be expected in the coming years [37].

2.5 Protein and Peptide Classification

One particular use case for machine learning with molecular fingerprints is protein classification. Protein classification is an important task in bioinformatics, since it allows researchers to correlate a protein’s structure with its biological function. Gaining knowledge about protein functions is an essential task in drug design [38]. With increasing computational capabilities, automated methods such as machine learning have found use in protein classification problems. Before applying machine learning models to the task of protein classifications, the respective compounds first have to be encoded into a suitable data format. Different approaches exist for encoding proteins, which can be roughly categorized into sequence-based and structure-based encodings [4]. In addition, various machine learning classifiers can be applied to the encoded data sets. These include established algorithms like random forests [39] or support vector machines (SVMs) [40], as well as deep learning methods such as deep neural networks (DNNs) [41]. As a result, there are numerous possible approaches to tackle the task of protein classification. Similarly, peptide classification plays an important role in finding future therapeutics. Specifically, cell-penetrating peptides (CPPs) enable novel approaches for intracellular delivery of therapeutically relevant molecules [42]. To improve screening performance, machine learning techniques can be applied to the task of peptide classification. Multiple studies have found success in this domain. Using SVMs for classification, Sanders *et. al.* [43] were able to confirm 100% of the classified CPPs to be penetrating by experimental validation. Similarly, Gautam *et. al.* [44] employed SVMs for CPP classification, achieving a maximum accuracy of 81.31% on independent data sets. Likewise, in a study published in 2018, Manavalan *et. al.* [45] reached a classification accuracy of 89.6% on independent data sets by combining four different machine learning algorithms.

3. Materials

The Materials chapter presents the data and software used in developing and evaluating the proposed encoding.

3.1 Data

The feature encoding developed as part of this work represents information about a molecule by encoding the neighborhoods around the carbon chain. Therefore, its main application is organic chemistry. Hence, the data sets used for evaluation can be any data set from the domain of organic chemistry, as long as structural information about the respective molecules is available. To minimize bias based on data set choice, the proposed encoding is evaluated on a variety of data sets. Seven of the eight data sets chosen for evaluation consist of peptides, with six containing cell-penetrating peptides (CPPs). CPPs are "generally peptides of less than 30 amino acids" [46, p. 196], making them an appropriate choice for the initial evaluation since their small size compared to other compounds helps keep computational complexity manageable. In addition, CPPs are an important topic in medical research because of their potential use in therapeutics [47]. This helps to highlight the practical application of the developed encoding method for real-world medical research.

To further avoid bias based on data set choice, two more data sets, not belonging to the family of CPPs, are chosen for evaluation. An overview of all data sets used for evaluating the proposed encoding can be found in Table 3.1.

The data sets are used as part of a binary classification task. In addition to the observations, each data set also contains a binary target vector. The class distribution of the respective target vectors vary for the different data sets, with the imbalance ratio (IR) ranging from 0.19 to 1. This helps in evaluating whether the proposed encoding can achieve good classification results even on class-imbalanced data. The IR is defined as the number of samples belonging to the positive class divided by the number of samples belonging to the negative class, i.e. $IR = \frac{m_P}{m_N}$ [48]. In an effort to ascertain if data set size has an effect on the quality of the prediction, the number of observations contained in each data set also varies.

3.1.1 Amino Acids

Proteins and peptides consist of amino acids. Hence, amino acids are often referred to as the building blocks of life [49]. Therefore, they are important to the data sets used for evaluation as part of this work. The data sets are initially available in the FAST-All (FASTA) format, introduced in detail in Section 3.1.3. The FASTA format denotes amino acid sequences using special 1-letter codes [5].

In addition to being integral to the data sets used in this work, the 20 proteinogenic amino acids are used to demonstrate the image representations that can be optionally generated by the proposed encoding. The structural data for the amino acids is taken from PubChem [50].

3.1.2 Data Sets

The data sets used for evaluation are compiled from a variety of literature. A detailed summary of the data sets’ original sources can be found in Table 3.1. Data sets prefixed by *cpp_* contain peptides with the common task of identifying cell-penetrating peptides. In their case, the property encoded in the target vector is whether or not the peptide is cell-penetrating. Of the remaining two data sets, the *ace_vaxinpad* data set also contains peptides, but its target vector represents information about stimulating antigen presenting cells (APCs). Lastly, the *hiv_protease* data set consists of data used for the prediction of human immunodeficiency virus (HIV)-1 protease cleavage sites. The properties encoded in the target vectors are represented by ones and zeros, corresponding to the presence or absence of the relevant property, respectively.

Identifier	Data set size	Imbalance ratio	Source
<i>ace_vaxinpad</i>	688	0.79	[51]
<i>cpp_cellppd</i>	1614	1	[44]
<i>cpp_cellppdmod</i>	1462	1	[52]
<i>cpp_cppredfl</i>	924	1	[53]
<i>cpp_kelmcpp</i>	1003	1	[54]
<i>cpp_mlcpp</i>	1903	0.63	[45]
<i>cpp_mlcppue</i>	374	1	[45]
<i>hiv_protease</i>	947	0.19	[55]

Table 3.1: Data sets used for evaluation.

3.1.3 Data Formats

The proposed encoding encodes information about the molecule by taking into account the neighborhoods surrounding its carbon chain. Because of this, the data sets have to contain two-dimensional structural information at the molecular level. Therefore, a data format capable of storing structural information is required. As defined in Section 1.2: Goals, the input format for the encoding should furthermore adhere to the standards of cheminformatics to enable compatibility with existing software tools. Thus, the SMILES data format is chosen as the input format for the encoding.

SMILES

The simplified molecular-input line-entry system (SMILES) format, initially introduced by David Weiniger in 1987 [56], is a line notation system for chemical compounds. It is used to represent two-dimensional chemical structures as one-dimensional character strings. Simplified molecular-input line-entry system (SMILES) makes "no attempt [...] to represent any three-dimensional arrangement of atoms" [56, p. 32]. Since the proposed feature encoding relies on two-dimensional structures alone, this does not present a limitation. Through six different rules, any molecule can be converted to a SMILES character string [56]:

- **Atoms** are simply represented by their element symbols.
- **Bonds** are denoted using a set of unique symbols {`-`, `=`, `#`, `:`}, with each symbol representing a specific type of bond.
- **Branches** are enclosed in parentheses.
- Additionally, the SMILES specification includes rules for representing **cyclic** and **disconnected** structures as well as **aromaticity**.

To demonstrate the rules by example, the phenol compound is represented by the SMILES string C1=CC=C(C=C1)O.

Because of its simplicity and its conceptual proximity to graph theory, SMILES has applications in database searching, substructure querying, and the prediction of chemical properties [56].

FASTA

The data sets used to evaluate the proposed encoding are initially available in the commonly used FASTA format. The FASTA format is the *de facto* standard format

for representing nucleotide or amino acid sequences [5]. Each nucleotide or amino acid is denoted by a single unique letter. Therefore, any sequence represented by the FASTA format is a one-dimensional character string. As stated in the previous section, the proposed encoding requires structural information of the molecules in order to generate the feature vectors. As a result, the data sets first need to be converted from FASTA into the aforementioned SMILES format. The tools used for this conversion are described in Section 3.2.

3.1.4 Data Structure

Initially, each data set is divided into two files, with the first containing all sequences belonging to the data set in FASTA format. The second file is a simple `.txt` file and contains the class vector of equal length. After converting the FASTA sequences into SMILES, the first file contains the SMILES representations.

The data sets are of varying sizes, ranging from 374 to 1903 observations. Three of the data sets have an imbalance ratio of $IR < 1$, meaning the frequency of positive and negative class labels are unequal.

After encoding the data with the proposed encoding, the resulting feature vectors are exported as `.csv` files for further processing. Each line in the `.csv` file contains one encoded molecule, with the columns corresponding to the numeric features generated by the encoding. The target variable remains unchanged in a separate `.txt` file.

Due to the parameter settings made available by the proposed encoding, six different permutations of the encoding are generated for each of the original data sets. Therefore, $8 * 6 = 48$ encoded data sets have to be evaluated. The different parameters that result in the permutations are discussed in detail in Chapter 4: Methods.

3.2 Software

The workflow for developing and evaluating the proposed encoding consists of multiple steps, including pre-processing, feature encoding, feature selection, and machine learning. These are discussed in detail in Chapter 4. Each step requires different specialized software tools optimized for the respective task. The following section will introduce the software tools employed as part of this thesis and highlight their use cases in the development pipeline.

3.2.1 Python

Significant parts of the encoding pipeline are implemented using Python 3.8.5 [57]. Because of its popularity in the domains of data science and machine learning, as well as interdisciplinary areas related to scientific computing [58], Python offers high compatibility with existing software tools commonly used for similar tasks from bioinformatics and cheminformatics. Additionally, due to its expressive syntax comparable to "executable pseudocode" [58, p. 10], the implemented methods are easy to understand even without explicit knowledge about computer programming, increasing the accessibility of the proposed encoding. Through its extensive external module library enabled by the open source license, Python furthermore offers the ability to publish the proposed encoding as a downloadable module. In addition, implementation overhead can be reduced by making use of available external modules and adapting their data structures.

3.2.2 Python Packages

All external Python packages used as part of this project can be found in Table 3.2. Some modules are highlighted in detail and their specific contribution to the project is discussed.

Package name	Version number	Documentation
Biopython	1.7.7	[59]
Matplotlib	3.3.0	[60]
NetworkX	2.4	[61]
NumPy	1.19.1	[62]
pandas	1.1.0	[63]
RDKit	2020.09.1	[64]
pysmiles	1.0.1	[65]

Table 3.2: Python packages used as part of the implementation.

Matplotlib

In addition to binary or numeric feature vectors, image representations are generated as part of the proposed encoding in an effort to increase accessibility and enable image space operations. For generating the images, the popular Matplotlib library is used [60]. The Matplotlib library provides methods for the creation of

images from numeric matrices. This simplifies the image generation step of the proposed encoding. Furthermore, color palettes can be specified for image generation, enabling custom coloration of elements in the images to achieve an intuitive visual representation of the encoded compounds.

NetworkX

In order to iterate the carbon chain and record the relevant neighborhoods represented by the proposed encoding, a graph-based approach is taken. The NetworkX package provides the data structures necessary for generating the graph representations. Moreover, standard graph algorithms and operations are exposed by the NetworkX application programming interfaces (APIs), reducing implementation complexity and streamlining the source code [61].

NumPy

As defined in Section 1.2: Goals, the output of the proposed encoding is a binary or numeric feature vector. To enable efficient numeric operations in generating the output feature vector, matrices are used as the primary data structure. The NumPy package enables computationally efficient matrix operations with a high-level language [62].

RDKit

RDKit is a popular cheminformatics and machine learning package written in C++ and adapted to Python. It offers functionality to support common cheminformatics tasks like two-dimensional and three-dimensional molecular operations, fingerprint generation, and data querying using substructure and similarity searching [64]. For the encoding developed as part of this thesis, RDKit is used in data pre-processing. As described in Section 3.1, the data sets must be converted from FASTA to SMILES in order to access the structural information necessary to generate the feature vectors. This data conversion step is performed using the RDKit Chem API.

3.2.3 R

For evaluating the proposed encoding, R 3.6.0 [66] is used. R is a popular environment for statistical computing. For this project, R is used for tasks related to machine learning, including feature selection, model training, and validation. These steps are enabled by the intuitive data structure offered by R’s `data.frame` function as well as an extensive external package library developed to support machine learn-

ing tasks. The external packages used as part of the implementation are introduced in the following section.

3.2.4 R Packages

With R being used for evaluating the proposed encoding, the external packages employed in the implementation support tasks such as computationally efficient machine learning model training and the extraction of performance metrics. An overview of all external packages used for this project can be found in Table 3.3.

Package name	Version number	Documentation
caret	6.0.86	[67]
doParallel	1.0.16	[68]
dplyr	1.0.2	[69]
e1071	1.7.4	[70]
ggplot2	3.3.2	[71]
ggpubr	0.4.0	[72]
ggthemes	4.2.0	[73]
janitor	2.0.1	[74]
MLeval	0.3	[75]
mlr	2.18.2	[76]
ranger	0.12.1	[77]
Rfast	2.0.1	[78]

Table 3.3: R packages used as part of the implementation

caret

The caret package [67] is used to implement the machine learning pipeline necessary for evaluating the proposed encoding. Caret offers the ability to easily combine different validation strategies, machine learning models, and performance metrics. Additionally, it allows for parallelized execution to improve computation speed and automates parameter tuning.

e1071 and ranger

The e1071 [70] and ranger [77] R packages provide the machine learning classifiers used as part of the evaluation pipeline. The SVM model is provided by the e1071 package, while the random forest model is provided by ranger.

4. Methods

The following sections establish the methodology used to implement and evaluate the proposed encoding. The methodology is divided into two parts. First, the encoding itself is introduced. This includes handling the input data, creating the necessary data representation, recording the relevant neighborhoods, transforming the features to their output format, and generating the image representations. The second part introduces the machine learning workflow used to evaluate the encoding.

4.1 Data Pre-Processing

Before developing the encoding and applying it to the data sets introduced in Section 3.1, the data format of the data sets needs to be converted from FASTA to SMILES. The SMILES format exposes information about the underlying molecular structure. Since the FASTA format denotes amino acid sequences, this structural information is only accessible after converting from FASTA to SMILES. Having access to the two-dimensional structure of the molecule enables the encoding algorithm to generate the intermediate data representation in the form of a molecule graph. Subsequently, the molecule graph can be traversed along the carbon chain to record the relevant neighborhoods of each carbon atom.

This step is not part of the encoding algorithm itself, since the algorithm is designed with SMILES as the input format. Rather, the pre-processing step is needed because of a constraint imposed by the choice of data sets.

The conversion is performed using the Python module RDKit introduced in Section 3.2. To improve code modularity, the data conversion step is implemented as a separate Python script. A high-level overview of the implementation workflow is depicted in Figure 4.1.

First, the amino acid sequences contained in the `.fasta` input file are parsed one-by-one. Each sequence is then converted to a molecule representation using the RDKit Chem API. Afterwards, the molecule is converted to a SMILES string and

appended to the output list. After iterating over all sequences in the input file, the output list containing all SMILES representations is saved as a separate `.smiles` file, with each line containing one molecule. This procedure is repeated for all eight evaluated data sets.

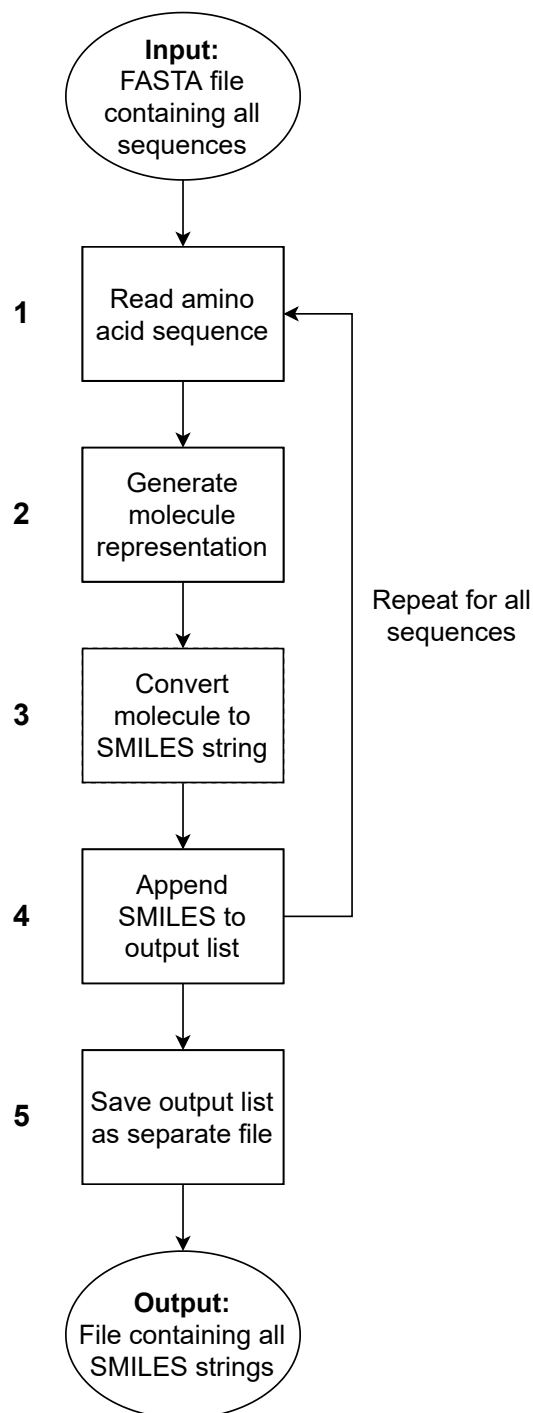


Figure 4.1: High-level workflow to convert FASTA sequences to SMILES strings.

4.2 Neighborhood Encoding

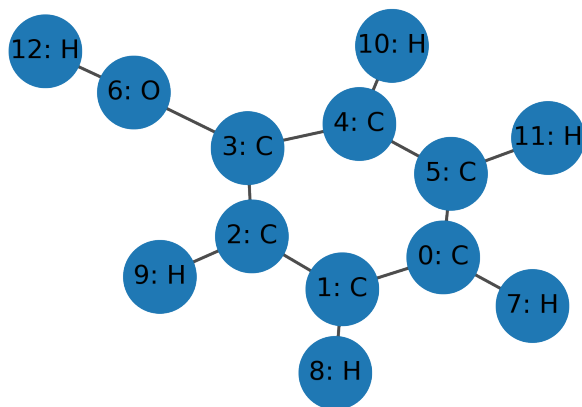
Generating the encoding according to the goals defined in Section 1.2 requires different steps depending on the parameters chosen for the encoding. To improve code modularity and allow for easier customization to suit the use case, the pipeline for generating the encoding is split into different parts. A high-level specification of the algorithm can be found in Figure 4.4. The different steps are introduced in detail in the following sections.

4.2.1 Molecule Graph

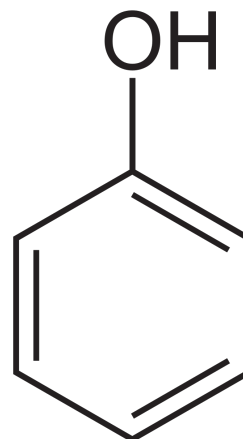
The first step in generating the encoding is translating the text-based SMILES input into a graph-based intermediate representation. This is done for multiple reasons. First, a graph-based representation simplifies the traversal of the carbon chain necessary to record the relevant neighborhoods. This helps improve computational performance and accessibility, since the use of graph representations for molecules, also called molecule graphs, is a common approach in cheminformatics [79]. Efficient handling of the molecule graph is further supported by external libraries that expose standard graph traversal approaches. Therefore, this step is implemented using Python’s NetworkX library introduced in Section 3.2. Secondly, by generating graph nodes for each atom of the molecule and labeling them with unique node identifiers, reproducibility can be ensured.

The algorithm also allows the user to plot the intermediate graph representation. An example of this plot using the phenol compound compared with its chemical structure depiction can be found in Figure 4.2. The plot is helpful for two reasons. Firstly, it gives the user insight into the molecule’s structure which might not be immediately apparent from the SMILES string. This can aid in identifying the optimal parameter settings for the encoding by revealing the overall structure of the compound. Secondly, the plot can be used as a visual validation for the SMILES input string. In this way, it serves as a plausibility check, a strategy commonly used in software development and engineering [80, 81]. The plot is generated using the NetworkX and Matplotlib Python libraries. Since generating the plot requires additional computational performance, the plotting step is implemented as an optional parameter.

In order to generate the molecule graph, the SMILES input string is read using the Python module pysmiles [65]. The result is an adjacency matrix which is transformed into the graph by the NetworkX package. Commonly, hydrogen atoms are



(a) Intermediate representation generated for phenol.



(b) Phenol structure [82].

Figure 4.2: Generated graph representation compared with the structure of phenol.

not explicitly included in the SMILES string notation, but instead are assumed from normal valence [56]. To record all the neighborhood information present within the molecule, hydrogen elements are explicitly filled in at the correct positions when parsing the SMILES input using the `explicit_hydrogen=True` parameter provided by the `pysmiles` package. This ensures all atoms of the original molecule are present for all following steps of the encoding algorithm.

Transforming the SMILES input string into the graph representation is deterministic for reproducibility. All atoms of the molecule, which are represented by nodes in the molecule graph, are numbered with a unique identifier. These identifiers are also assigned deterministically. In addition to the identifiers, the nodes of the molecule graph store the type of element they represent using its element symbol from the periodic table. The element labels are required later on to generate the feature vectors.

To avoid redundant information not required as part of the encoding, the edges of the molecule graph do not store any additional information aside from the nodes they connect. Specifically, the bond types between the atoms are not included in the molecule graph. By design, this information is irrelevant for the proposed encoding and can therefore be safely omitted to improve the memory usage of the algorithm. The procedure described up to this point covers steps 1, 2, and 3 of the workflow depicted in Figure 4.4.

4.2.2 Molecular Neighborhoods

The neighborhoods along the carbon chain are the basis of the feature encoding. Therefore, the identification of those neighborhoods is the main challenge in de-

veloping the encoding algorithm. Additionally, as outlined in Section 1.2: Goals, the encoding algorithm needs to provide parameters to control the neighborhoods recorded depending on the size and structure of the underlying compound.

The first step for recording the neighborhoods, labeled as step 4 in Figure 4.4, consists of walking along the carbon chain of the molecule. To achieve this, the molecule graph is traversed iteratively. First, the nodes of the molecule graph are filtered according to their element symbol. Only nodes with the label **C**, i.e. carbon atoms, are included in the filtered list. Each carbon node is included exactly once. The filtered nodes are then sorted in ascending order by their unique node identifier. The sorting step ensures that the order in which the nodes are traversed is deterministic for reproducibility. Determinism is ensured because the unique identifiers are assigned deterministically, as described in the previous section. The resulting filtered and sorted list is iterated with a loop.

Since this list serves as the basis for the iteration, cycles in the molecular structure, for instance the aromatic cycles present in some amino acids, are properly handled. Like all other nodes in the molecule graph, the atom nodes that form the cycles also possess unique node identifiers and are included only once in the filtered and sorted list. This avoids infinite loops along the cyclical substructures during the iteration. For each carbon node, its neighborhood is recorded. Initially, only its immediate neighbors are saved. The immediate neighbors include all nodes connected directly by an edge to the respective carbon node. Aside from the immediate neighbors, additional hierarchy levels of neighborhood can be saved. The additional hierarchy levels are defined as the immediate neighbors of all nodes belonging to the previous level. For instance, the second-level hierarchy includes all nodes with a direct connection to any node from the first-level hierarchy. To avoid redundancy in the encoded information, an additional filter is applied when recording more than one hierarchy. This filter excludes nodes containing carbon atoms, since their neighborhoods are recorded as part of the main iteration. The number of recorded hierarchies, also called the depth, can be controlled using the algorithm’s `max_hierarchies` parameter. Up to three hierarchy levels can be recorded. From a technical standpoint, the limit of three hierarchy levels is arbitrary and could be extended to any positive integer. However, as outlined in Section 1.1: Background and Motivation, the neighborhoods surrounding the carbon chain are the main focus of the proposed encoding. Recording a greater number of neighborhoods would contradict this initial motivation. Therefore, a limit of three hierarchy levels is defined. The optimal parameter setting of the depth depends on the size and structure of the molecule and has to be specified by user input. The plot of the intermediate graph represen-

tation can aid in identifying the optimal depth by revealing the molecule’s overall structure. After recording all desired hierarchies for the respective carbon atom, all neighboring nodes are combined into a single list and saved.

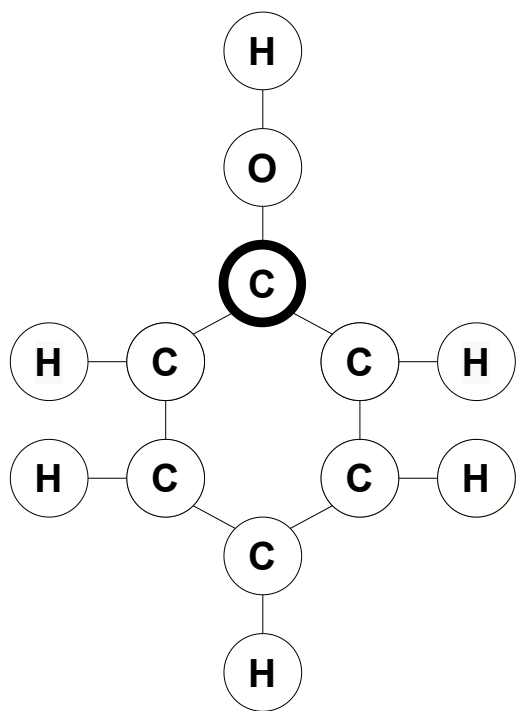
The data structure used for saving the neighborhoods is a Python dictionary. Each carbon node serves as a key in the dictionary. The list containing its recorded neighborhood is saved as the dictionary value under the respective key. Only the element symbols belonging to the nodes of the neighborhood are saved. The list of element symbols is, by nature of the iteration, automatically sorted according to the unique node identifiers. This ensures that the feature vectors are deterministic across multiple runs of the encoding.

Recording the neighborhood is repeated for all carbons in the chain. The resulting dictionary includes the neighborhoods for all carbon atoms. Before returning the result, the dictionary is transformed to a data frame structure. This simplifies the operations in the following steps of the algorithm. Since the neighborhoods up to this point are recorded using the neighboring elements’ symbols, the generated representation is a categorical data frame. This presents additional challenges which are discussed in the next section.

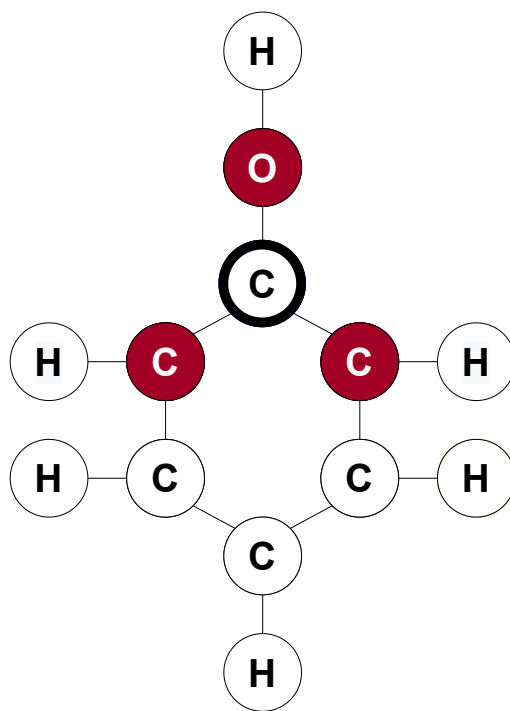
Additionally, the resulting categorical data frame may include missing values depending on the structure of the encoded compound. As described previously, carbon nodes are specifically excluded when recording more than one hierarchy level in order to avoid redundancy in the encoded information. Because of this, it can occur that the number of recorded atoms in the neighborhoods along the carbon chain is unequal. If this is the case, the data frame is automatically filled with missing values in the relevant positions in order to preserve the overall data structure. The missing values are properly handled in the following step of the encoding algorithm. A visual example for recording the neighborhoods using the phenol compound can be found in Figure 4.3. Note that the depicted step is only a snapshot of the whole algorithm, meaning only the neighborhood of the specific carbon node C_3 is being recorded in the figure. The data frame resulting from recording all neighborhoods of phenol can be found in Table 4.1. For this example, only first- and second-level hierarchies are recorded.

C_0	C_1	C_2	C_3	C_4	C_5
C	C	C	C	C	C
C	C	C	C	C	C
H	H	H	O	H	H
C	C	C	C	C	C
-	-	-	H	-	-

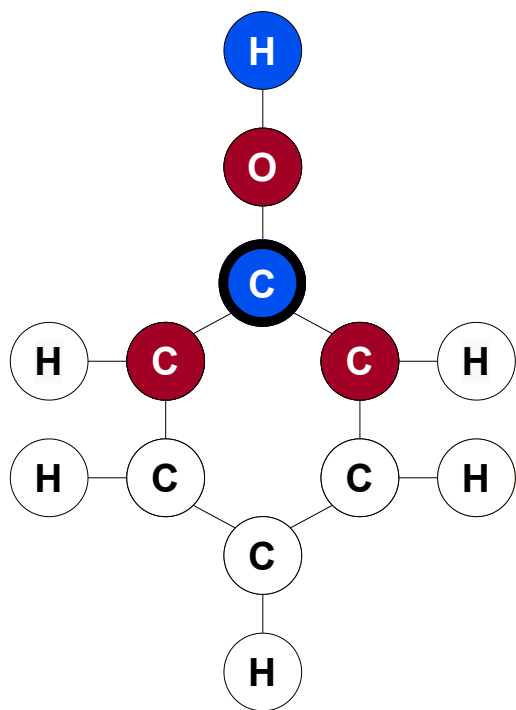
Table 4.1: Neighborhoods of phenol using first- and second-level hierarchies.



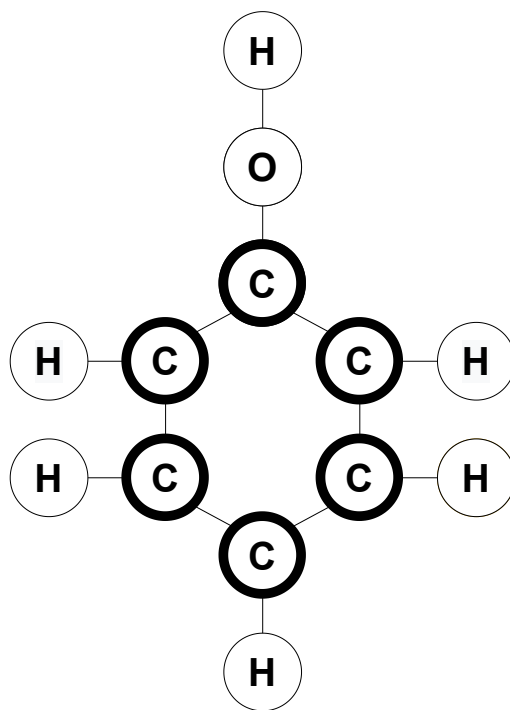
(a) Step 1: The iteration reaches the highlighted carbon element.



(b) Step 2: Record first-level hierarchy (highlighted in red).



(c) Step 3: Record second-level hierarchy (highlighted in blue).



(d) Step 4: Repeat recording of neighborhoods for remaining carbons.

Figure 4.3: Recording neighborhoods along the carbon chain for phenol. The visualized iteration step corresponds to carbon node C_3 in Table 4.1.

4.2.3 Feature Transformation

The initial result from walking the carbon chain and recording the neighborhoods around the carbon nodes is a categorical data frame. This poses several problems. First, as defined in Section 1.2: Goals, the encoding must allow for the application of distance and similarity measures to support tasks such as similarity searching. A thorough introduction to this topic as well as an overview of common metrics and techniques was given in Chapter 2. Most of the available metrics can however only be applied to binary or numeric vectors. Secondly, generating the image representations for the encoding requires numeric data to assign as pixel values. Thirdly, depending on the structural properties of the underlying compound, the initial categorical data frame may include missing values. To solve these problems as part of the encoding pipeline, the categorical neighborhoods are converted to either binary or numeric feature vectors using two different approaches.

Binary Transformation

The first strategy for transforming the categorical neighborhoods is a binary transformation using dummy encoding. Dummy encoding is a common approach in statistics and machine learning with applications in regression analysis [83], text mining [84], and bioinformatics [4]. With dummy encoding, categorical variables can be represented with binary values. Commonly, 0 and 1 are used as binary values, with 0 representing the absence of a certain categorical value and 1 its presence [85]. In the context of the proposed encoding, 0 and 1 encode the absence or presence of an atom in the respective neighborhood. As a side effect, missing values in the categorical data frame are filled with the value 0 during binary transformation. Because the value 0 represents the absence of information, this approach does not distort the resulting feature vector.

In line with the previous example using the phenol compound from Table 4.1, the transformed binary data frame can be found in Table 4.2.

After transforming the encoding from categorical to binary using dummy encoding, the feature vectors can be represented as bit strings. Therefore, the distance and similarity measures introduced in Chapter 2, for example the Tanimoto coefficient, can be calculated between two encoded molecules.

C_0C	C_0H	C_1C	C_1H	C_2C	C_2H	C_3C	C_3H	C_3O	C_4C	C_4H	C_5C	C_5H
1	0	1	0	1	0	1	0	0	1	0	1	0
1	0	1	0	1	0	1	0	0	1	0	1	0
0	1	0	1	0	1	0	0	1	0	1	0	1
1	0	1	0	1	0	1	0	0	1	0	1	0
0	0	0	0	0	0	0	1	0	0	0	0	0

Table 4.2: Neighborhoods recorded for phenol using first- and second-level hierarchies after binary transformation.

CPK-based Discretization

As an extension of the binary transformation using dummy encoding introduced in the previous section, the proposed encoding also offers discretization based on the Corey–Pauling–Koltun (CPK) coloring scheme. For discretization, the features are transformed to integers ranging from 0 to 16. Each number uniquely encodes one element or element group. The range of numbers is based on the CPK coloring system. The CPK system, first introduced in 1953 [86] and extended in 1965 [87], is a system used for modelling chemical compounds. CPK modelling is widely used in chemistry software toolboxes, for instance in MolScript [88] and MBT [89].

In addition to feature transformation, the CPK mapping is also used later on in the encoding pipeline for image generation. The color assignments for different types of elements in the generated images follow the CPK colors. Since there is no exact definition of color mapping for the CPK system, the proposed encoding uses common assignments found across molecule kits. This increases readability and accessibility for users with prior domain knowledge. The complete mapping can be found in Table 4.4. It should be noted that even though some of the elements listed in Table 4.4 are redundant for the specific use case of organic chemistry, they are still included in the mapping in order to adhere to the common standards used across molecule kits.

Aside from values representing specific elements or elements groups, two additional integers are included in the mapping. Similarly to the dummy encoding, the number 0 encodes the absence of an atom at the respective position. In order to contrast with the white color used to represent hydrogen, a light grey color is assigned to the value 0. Lastly, 1s are used for all other elements not specifically part of the CPK mapping.

Using the phenol example from Table 4.1, the data frame transformed with CPK-based discretization can be found in Table 4.3.

C_0C	C_0H	C_1C	C_1H	C_2C	C_2H	C_3C	C_3H	C_3O	C_4C	C_4H	C_5C	C_5H
3	0	3	0	3	0	3	0	0	3	0	3	0
3	0	3	0	3	0	3	0	0	3	0	3	0
0	2	0	2	0	2	0	0	5	0	2	0	2
3	0	3	0	3	0	3	0	0	3	0	3	0
0	0	0	0	0	0	0	2	0	0	0	0	0

Table 4.3: Neighborhoods recorded for phenol using first- and second-level hierarchies after CPK-based discretization.

Element or element group	Element symbol(s)	Number	Color
Hydrogen	H	2	
Carbon	C	3	
Nitrogen	N	4	
Oxygen	O	5	
Fluorine, Chlorine	F, Cl	6	
Bromine	Br	7	
Iodine	I	8	
Noble gases	He, Ne, Ar, Xe, Kr	9	
Phosphorus	P	10	
Sulfur	S	11	
Boron	B	12	
Alkali metals	Li, Na, K, Rb, Cs, Fr	13	
Alkaline earth metals	Be, Mg, Ca, Sr, Ba, Ra	14	
Titanium	Ti	15	
Iron	Fe	16	
Absence of element	-	0	
Other elements	-	1	

Table 4.4: Mapping used for discretization based on the CPK system.

The different methods for feature transformation can be controlled using the `binary_encoding` parameter provided by the encoding algorithm. When set to `True`, the categorical features are transformed to binary using dummy encoding. This is the default setting. When set to `False`, CPK-based discretization is applied. Feature transformation represents steps 6 and 7 of the workflow depicted in Figure 4.4 and concludes the application of the encoding on a single molecule. The

resulting data frame, which now contains either binary values or integers ranging from 0 to 16, is returned by the algorithm. This data frame is the molecular fingerprint generated for the input molecule.

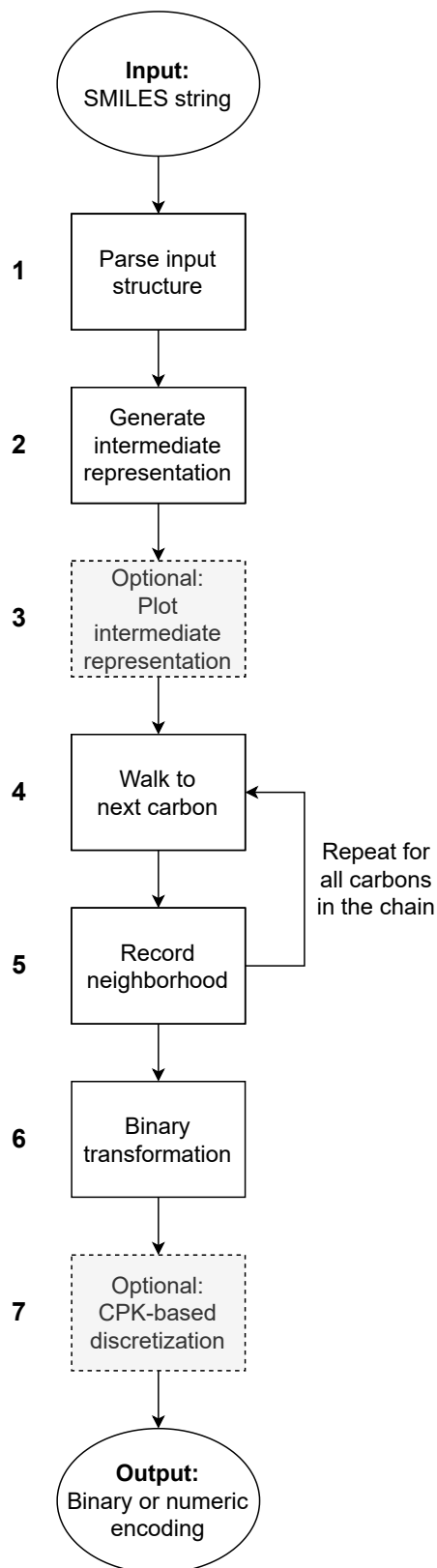


Figure 4.4: High-level workflow to generate the encoding for a single molecule.

4.2.4 Padding the Encoding

The previously described steps of the algorithm assume the encoding is applied to a single molecule only. However, for most use cases, data sets consisting of a large number of molecules are encoded at once. This presents an additional challenge. The number of features generated by the encoding depends on the number of carbon nodes along the carbon chain as well as the structure of their neighborhoods. Therefore, differences in molecular structure often lead to mismatched dimensions in the output feature vectors for different compounds in a data set. This makes accurate comparison of encoded molecules impossible.

Other common encodings also face this issue. As described in Chapter 2, topological molecular fingerprints employ the use of a hashing function to ensure equal feature length. However, hashing functions display additional downsides. Firstly, hashing functions introduce another layer of abstraction to the encoding. Secondly, since the number of output bits has to be finite, bit collision can occur by hashing different encoded features to the same bit. This results in a further loss of information [5]. Because of these reasons, a different strategy for ensuring equal feature length is implemented in the proposed encoding.

To solve this issue, an additional step is included in the encoding pipeline when applying the encoding to more than one molecule. As described in Section 4.2.3, the value 0 in the output feature vector encodes the absence of information at the respective position. Therefore, 0s can be used for padding without distorting the meaning of the feature vector. Two different padding strategies are implemented in the encoding pipeline. The strategy to use can be toggled with the `center_encoding` Boolean parameter.

To optimize computational performance, the padding step is implemented using matrix operations with the NumPy Python package. For padding, the numeric data frames resulting from the transformation step described in the previous section are converted to matrices. In a first step shared across both implemented padding strategies, each matrix m_i is then squared to dimensions $n_i \times n_i$ in order to ensure an equal number of rows and columns. To determine n_i , i.e. the target dimension of each individual matrix, each matrix m_i is first flattened in column-major order to a one-dimensional array l_i . Then, n_i^2 is determined by calculating the next perfect square of l_i 's length:

$$n_i^2 = \left(\lfloor \sqrt{\text{len}(l_i)} \rfloor + 1 \right)^2$$

To coerce the matrix representation into the correct dimensions, zeros are appended to the one-dimensional array l_i until its length equals n_i^2 . Lastly, l_i is transformed back into matrix form, its shape now being $n_i \times n_i$. During this procedure, the maximum dimension across all matrices in the data set, n_{max} , is stored. The n_{max} value serves as the baseline value for the padding step.

After ensuring each matrix is in square shape, one of the two available padding strategies is applied depending on the `center_encoding` parameter. The two strategies are visualized in Figure 4.5. If the centering strategy is selected, each encoding in matrix form is centered in a larger matrix of size $n_{max} \times n_{max}$. If the shifting strategy is chosen, each encoding in matrix form is shifted to the top left of a matrix of size $n_{max} \times n_{max}$. In both cases, the missing values are padded with zeros. After ensuring all feature matrices are padded to same dimensions, they are transformed back into their original data frame type and returned by the algorithm. Optionally, the feature vectors can be saved as a `.csv` file to a user-specified location.

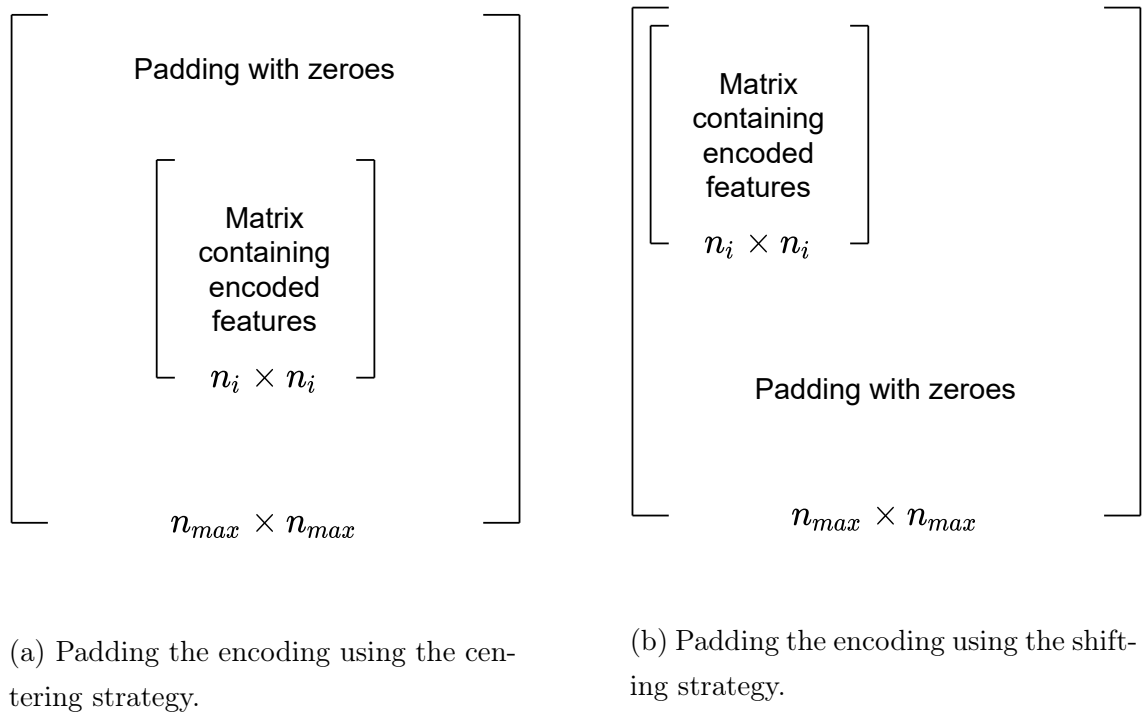


Figure 4.5: Strategies for padding the encoding of size $n_i \times n_i$ to match $n_{max} \times n_{max}$.

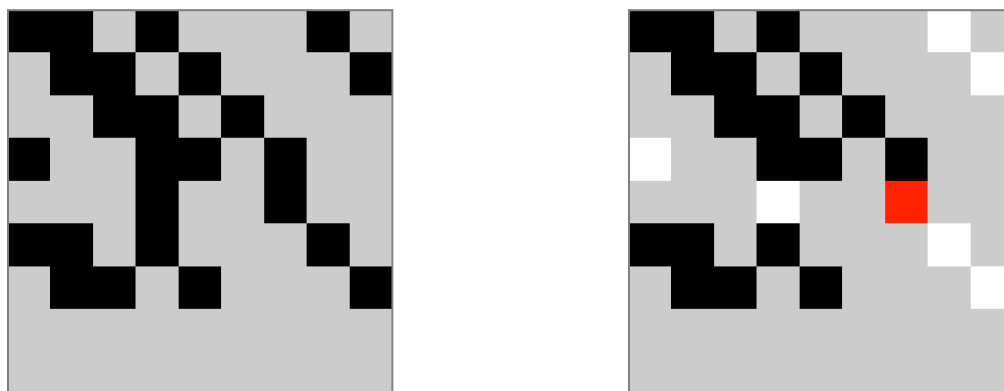
4.2.5 Image Generation

As defined in Section 1.2: Goals, image representations of the encoding are generated in addition to the binary or numeric feature vectors. This increases accessibility and enables image space operations. Depending on the encoding parameters selected, different image representations are generated. If the transformation step described in Section 4.2.3 is performed using the binary setting, the resulting images are drawn with a binary color palette. If CPK-based discretization was applied, the images follow the CPK coloring scheme. Due to the padding step described in the previous section, the generated images are always in square shape. Furthermore, the padding step also ensures that the generated images for all molecules belonging to the same input data set have equal dimensions.

To generate the images, the binary or numeric data frames resulting from the previous step of the encoding pipeline are converted to pixel values. For this, each value is mapped uniquely to a color using the color’s hexadecimal code. To account for both feature transformation strategies, two color palettes are defined with the Python module Matplotlib. The first color palette contains the color mapping for binary feature vectors. In this case, two colors are used. The value 0 is represented by light grey, while the value 1 is represented by black. The choice to use light grey instead of white for the value 0 is made to keep the color palette consistent with the second transformation strategy. The second color palette contains the color mapping for CPK-discretized feature vectors which follows the system outlined in Table 4.4.

Example image representations for the phenol compound using both binary and CPK color palettes can be found in Figure 4.6. To further highlight the versatility of this method on a larger molecular structure, Figure 4.7 contains the CPK-colored image generated for sequence 30 of the *ace_vaxinpad* data set.

The generated images are saved as `.png` files to a user-specified location. Since image generation introduces additional computational overhead, this step is implemented as an optional parameter. Image generation concludes the implementation of the encoding pipeline.



(a) Phenol with the binary color palette.

(b) Phenol with the CPK color palette.

Figure 4.6: Image representations of the encoding for phenol. Generated based on the feature vectors found in Table 4.2 and Table 4.3, respectively.

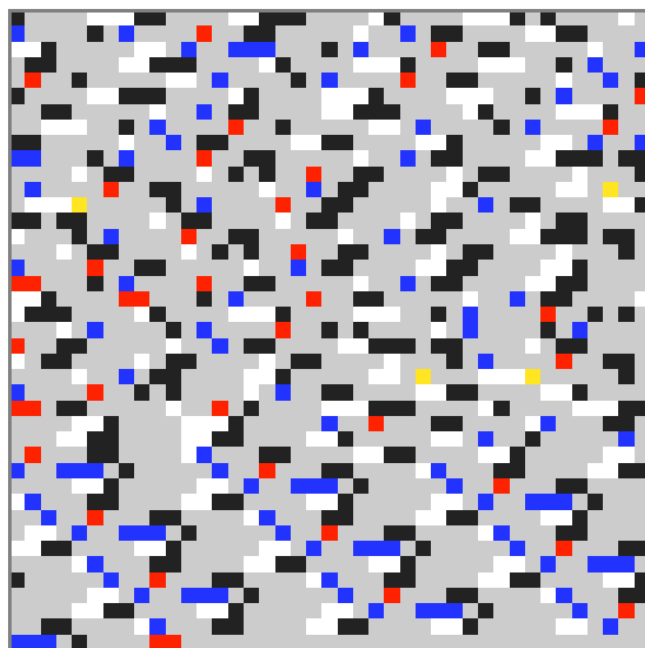


Figure 4.7: CPK-colored image representation generated for sequence 30 of the *ace_vaxinpad* data set.

4.3 Evaluation

The following sections deal with evaluating the proposed encoding using the data sets introduced in Section 3.1. Evaluation represents the second major part of the work presented in this thesis. For evaluation, a separate pipeline is implemented using the R programming language. The evaluation pipeline serves to answer two main questions. First, whether or not the proposed approach can produce a suitable feature encoding for molecules. Second, which combination of recorded hierarchy depth and feature transformation strategy achieves the best results on the tested data sets.

By combining the three different hierarchy level settings with the two feature transformation strategies, namely binary transformation and CPK-based discretization, six different permutations of the encoding have to be evaluated for each data set. Since eight initial data sets are used, this results in a total of 48 different encoded data sets. The difference between the two padding strategies introduced in Section 4.2.4 are not evaluated, since their application mainly has an effect on the generated image representations. Instead, the centering strategy is used for all encoded data sets.

As defined in Section 1.2: Goals, the proposed encoding is evaluated on a task closely related to real-world applications. Therefore, machine learning is used. Each data set is utilized in a binary classification task. The different aspects of the machine learning pipeline are introduced in the following sections, including feature selection, model choice, validation strategy, and evaluation metrics. By combining multiple feature selection techniques and classification models, bias due to mathematical or stochastic effects is minimized.

A high-level overview of the implemented pipeline can be found in Figure 4.8.

Additionally, to ensure the performance metrics calculated during the evaluation process are reproducible, a random seed is set for all steps of the evaluation pipeline using the `set.seed` R function.

4.4 Dimensionality Reduction

The first step of the evaluation pipeline is dimensionality reduction. Depending on the parameter settings used and the size and structure of the encoded compounds, the encoding can produce very high-dimensional feature vectors. For the evaluated data sets, the ratio of features to observations ranges from 2 for the *hiv_protease*

data set to 36 for the *cpg_mlcpgue* data set.

This can be attributed to multiple factors. Firstly, the number of features generated by the encoding for any molecule is determined by the length of the molecule’s carbon chain. Hence, longer carbon chains produce more features. Additionally, the size and structure of the recorded neighborhoods also influences the number of features. This holds especially true when recording more than one hierarchy level. Moreover, during the transformation step from categorical neighborhoods to binary or numeric feature vectors, dummy encoding is utilized. Since each categorical value results in an additional feature, dummy encoding inherently increases dimensionality. Lastly, the padding step also increases feature size by adding zeros to the encoding according to one of the strategies depicted in Section 4.2.4.

In order to retain the maximum informative value of the encoding and keep computational overhead to a minimum, dimensionality reduction is implemented as part of the evaluation pipeline instead of the encoding itself. This way, different methods for dimensionality reduction can be applied without having to re-encode the data sets. Dimensionality reduction has been shown to have multiple advantages. For one, including irrelevant features during model training can lead to decreased prediction accuracy [90]. Therefore, these features should be filtered out before training any classifier. Furthermore, decreasing the number of features leads to better computational performance [91]. This is especially important for this project, since a total of 48 different encoding permutations are generated from the available data sets. In addition, identifying important features and interpreting them in the context of the specific task can help in gaining insight about the underlying compounds.

4.4.1 Feature Pre-Processing

The first step for dimensionality reduction consists of feature pre-processing. During feature pre-processing, simple statistical operations are performed to reduce the number of initial features. Feature pre-processing consists of two steps. First, constant features are filtered out. Second, correlations between features are investigated and strongly correlated features are removed.

Constant Features

The first statistical-based method applied for dimensionality reduction is the removal of constant features. Constant features are those that show the same value across all observations. Constant features have a variance of $\sigma^2 = 0$ and therefore "cannot help discriminate instances from different classes" [92, p. 19].

The feature transformation step from categorical to either binary or numeric as well as the padding step introduce many constant-zero features. This makes an initial filter-based approach to filter out constant features especially useful for reducing dimensionality.

Correlation

The second statistical filter applied to the encoded data sets is a correlation-based approach. Removing correlations reduces redundancy in information between the selected features [93]. Multiple approaches to apply this filter exist based on different ways to measure correlation [94]. For this project, all pairwise correlations between features are calculated using Pearson’s correlation coefficient. This is enabled by the transformation step described in Section 4.2.3, which transforms the categorical neighborhoods to binary or numeric feature vectors.

Correlation coefficient values between 0.7 and 1.0 indicate a strong relationship between variables [95]. For this work, the threshold is set at 0.8. Therefore, all features with a correlation coefficient value of $r \geq 0.8$ are removed from the encoded data.

4.4.2 Feature Selection

After pre-processing the features using simple statistical filters, more complex feature selection techniques are applied to the encoded data sets. In order to minimize bias based on the feature selection method, different categories of feature selection are utilized. The categories along with their specific methods applied as part of this project are explored in the following sections.

Projection: Principal Component Analysis

The first category of dimensionality reduction discussed as part of this work are projection-based methods. Projection-based methods are enabled by the Johnson-Lindenstrauss lemma, stating that a set of points can be embedded in a much lower-dimensional space without distorting the pairwise distances between any two points by more than a factor of $1 + \epsilon$ [96]. Projection-based dimensionality reduction methods have numerous applications, especially for very high-dimensional data sets like images or text data [97]. Popular projection methods include principal component analysis (PCA) [98], multidimensional scaling (MDS) [99], and t-distributed stochastic neighbor embedding (t-SNE) [100]. For the work presented in this thesis, PCA is used.

The decision to use PCA in favor of other projection methods is made for two

reasons. First, calculation of PCA is computationally faster than other common projection methods [101]. Optimizing computational time of all steps in the evaluation pipeline is especially necessary for this work, since a total of 48 encoded permutations have to be evaluated. Second, in contrast to other common projection methods, a previously calculated PCA can easily be applied to out-of-sample data. The method to achieve this and the benefits thereof are further discussed in Section 4.4.3.

Dimensionality reduction via PCA is carried out by projecting the original data points into a new coordinate system with the goal of identifying the data set's principal components. The principal components are constructed in such a way that the first principal component explains the largest variance in the data set. The second principal component is defined as being perpendicular to the first, while accounting for the second largest variance across the data set. The same condition holds true for all further principal components. Thus, PCA is a linear projection method. The principal components are determined by calculating the eigenvalues and eigenvectors of the covariance matrix of the original input data [98].

After identifying the principal components in this manner, the first n principal components are chosen, where n is any positive integer less than or equal to the dimension of the original data. Therefore, the original data points are projected into an n -dimensional space.

Different strategies for choosing the optimal n exist. Commonly, n is determined "by making a scree plot and looking for [...] an 'elbow'" [102, p. 918]. Alternatively, Kaiser's rule can be applied to select all principal components with an eigenvalue of $\lambda > 1.0$ [103]. Furthermore, the method of total explained variance can be used to determine how many principal components are necessary to explain a predefined threshold of total variance [104]. For the work presented in this thesis, the method of explained variance is used with a threshold of 0.8.

Filter: Variable Importance

The second feature selection method used in the evaluation pipeline is variable importance. Variable importance is a filter-based feature selection approach, meaning it is applied independent of any trained model [92]. Multiple implementations exist based on different importance measures. The implementation used as part of this project uses the Gini impurity measure. Gini impurity is commonly used as a split criterion in decision tree learning [105]. It is defined as:

$$1 - \sum_{i=1}^C p_i^2$$

with C being the total number of observed classes and p_i the probability of a data point belonging to the respective class [106]. The variable importance is calculated for each feature individually.

To select the relevant features, they are first ranked based on their calculated variable importance. Then, the n best ranking features are selected. For this work, the number of selected features n is based on the concept of diminishing returns and is calculated using the elbow method. The elbow method is commonly utilized in cluster analysis to determine the optimal number of clusters [107]. Here, n is determined by identifying the elbow point of the curve that results from plotting the number of features against their explained variable importance.

Similar to PCA, the variable importance filter offers fast computational performance and allows for out-of-sample application.

Wrapper: Recursive Feature Elimination

Lastly, a wrapper-based feature selection method is employed to reduce dimensionality of the encoded data sets. In contrast to filter-based methods, wrapper-based approaches act in combination with a model to optimize the selected features [92]. Depending on the specific approach, different strategies for optimizing the feature set are available. For the work presented in this thesis, recursive feature elimination (RFE) is used. With RFE, the underlying model is trained multiple times. During each iteration, a subset of features is removed from the data set and the model is re-trained using only the remaining features. This process is repeated until the best feature set is found [92].

The number of features to retain during each training iteration is fixed and has to be specified beforehand. As part of this project, a function to dynamically calculate the number of retained features is implemented. With the number of observations as the input, the function returns a vector containing the integer values corresponding to the number of features to retain during each training iteration. The specific rules to determine this vector are displayed in Table 4.5.

For example, for the *cpp_mlcppue* data set containing a total of 374 observations, the resulting vector contains the following values:

[5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 75, 100, 125, 150, 175, 200, 225, 250, 300, 350].

The decision to base the number of retained features on the number of observations is based on research by Hua *et al.* [90]. The authors investigated the optimal number of features as a function of sample size and found that for uncorrelated data, "the optimal feature size is around $n - 1$ " [90, p. 1511], where n is the number of observations. Since correlations are removed from the encoded data sets during

feature pre-processing, the findings by Hua *et al.* are applicable.

Even though RFE introduces high computational complexity, it is included in the evaluation pipeline in order to represent wrapper-based approaches.

Interval	Step size
$[5, \min(50, n))$	5
$[50, \min(250, n))$	25
$[250, \min(500, n))$	50
$[500, \min(1000, n))$	100
$[1000, n)$	250

Table 4.5: Rules to determine the number of features to retain during RFE; n is number of observations.

4.4.3 Feature Selection and Validation

The evaluation pipeline is designed to minimize bias based on data set, feature selection approach, and machine learning model choice. To achieve this, two different validation strategies are combined. These are discussed in further detail in Section 4.5.1. For feature selection specifically, this requires calculating the optimal features only on the training subset. However, in order to evaluate the trained models using the hold-out data, both subsets must be in the same feature space. Therefore, the employed feature selection approaches must support being applied to out-of-sample data.

For PCA, this additional challenge can be solved by using the rotational matrix R . The rotational matrix R , in the context of PCA also called *loadings*, is calculated from the eigenvalues and eigenvectors of the covariance matrix during the initial application of PCA on the training data. In order to project the hold-out data into the same coordinate system and therefore the same feature space as the training data, it is multiplied with the previously calculated rotational matrix.

For the remaining feature selection approaches, namely variable importance and RFE, transforming the hold-out data into the same feature space does not require any advanced computations. Since both feature selection approaches simply select the relevant features without additional transformations, the same selection can be applied to the hold-out data.

4.5 Machine Learning

After selecting the optimal feature subsets according to the strategies introduced in the previous section, the machine learning models are trained. In order to minimize bias based on model choice, two different classifiers are trained. As defined in Section 3.1, the classifiers are used in a binary classification task to predict the different biological properties encoded in the data sets' target vectors.

Three main requirements are taken into account for choosing the machine learning models. Firstly, the models should already be established and used in a variety of research for different tasks. Since the proposed encoding is a novel approach, an established model is necessary to ensure the achieved results can be attributed to the proposed encoding instead of a model specifically optimized for the relevant task. Secondly, the model needs to be able to handle very high-dimensional data. As described previously, the proposed encoding tends to generate very high-dimensional feature vectors. With the feature selection step kept entirely optional, the classification models need to solve a $d \gg n$ problem. Lastly, the models should be reliant on different internal methodology to minimize bias based on the underlying mathematical aspects. Specifically, a linear and a non-linear classifier are used.

For these reasons, a random forest model and a linear weighted support vector machine (SVM) model are selected.

Random Forest

Random forests are part of the family of ensemble classifiers and can be used to solve regression and classification tasks. Predictions are made by aggregating the individual decisions from a large number of independent decision trees [108]. Since decision trees are considered non-linear classifiers, random forest are also able to separate data non-linearly. Multiple implementations for random forests exist, based on different decision tree types, internal validation strategies, and methods for aggregating the individual votes. For this work, the implementation from the *ranger* R package is used, which is especially suited for high-dimensional data [77].

Support Vector Machine

The second model used is a linear weighted support vector machine. Like random forests, SVMs can be used to solve regression and classification tasks. SVMs try to fit a hyperplane in an n -dimensional space that optimally separates the input data into its inherent classes. This is achieved by maximizing the margin between the hyperplane and the data points associated with the respective classes [109]. Linear

and non-linear implementations exist. For this work, a linear implementation from the *e1071* R package is used [70]. Since three of the data sets used for evaluation have an imbalance ratio of $IR < 1$, the SVM implementation is extended to include class weights [110].

4.5.1 Validation

Choosing the right validation strategy can have significant effects on the overall quality of the machine learning process. Firstly, in order to properly confirm the results and avoid introducing biases, it is necessary to calculate the final performance metrics only on previously unseen data. Secondly, training the model with a separate subset of data minimizes the risk of overfitting the model to the training data [111]. Additionally, to optimize the model's hyperparameters, the training process needs to be repeated multiple times with different parameter settings.

Depending on the size and structure of the input data set, different validation strategies can be employed. Commonly, hold-out validation or cross-validation (CV) are used. These basic methods can be modified to better suit the specific use case. For instance, CV can be extended by nesting multiple CV runs. This is referred to as nested cross-validation [112].

For the evaluation pipeline implemented as part of this work, two different validation strategies are combined. Initially, the input data is divided into training and testing subsets. Because three of the eight evaluated data sets are class-imbalanced, a stratified hold-out split is applied. Using a stratified split ensures the percentage of samples belonging to each class is preserved in both subsets [113]. The split ratio is 80% : 20%, meaning the training subset includes 80% of the original observations while the test set, also referred to as the hold-out set, includes 20%. The hold-out set is kept completely separate from all following operations and is only used to calculate the final performance metrics of the trained models. Only the training subset is used for feature selection and model training.

During model training, the hyperparameters of the random forest and SVM model are optimized. For the random forest, the hyperparameters consist of the number of variables to split at in each node, the split rule, and the minimum node size [77]. For the SVM model, the cost value of the regularization term and the class weights are optimized [70]. To properly tune the hyperparameters of the trained classifiers, a 10-fold stratified cross-validation is used for training. During each of the ten iterations (i.e. the folds), different data subsets are used to train and subsequently validate the model. Since this process is repeated ten times, the split ratio in each

iteration is 90% training data to 10% validation data. Choosing the optimal hyper-parameters for each model is automated with the *caret* R package. Because each CV fold is independent from the others, the model training step can be parallelized. Specifically, the training for each fold is performed on a separate CPU core using the *doParallel* R package.

4.5.2 Metrics for Evaluation

In order to properly quantify the performance of the trained classifiers, performance metrics suitable for binary classification tasks have to be calculated. The final performance metrics are determined only with the hold-out subset. They are based on the confusion matrix of the trained classifiers. The confusion matrix contains the counts of true positives (TPs), false positives (FPs), true negatives (TNs), and false negatives (FNs). From these, additional metrics are calculated [114]:

- True positive rate (TPR) (also: recall, sensitivity): $TPR = \frac{TP}{TP+FN}$
- False positive rate (FPR): $FPR = \frac{FP}{FP+TN}$
- Precision: $Precision = \frac{TP}{TP+FP}$

With the TPR and FPR, the receiver operating characteristic (ROC) curve can be plotted. ROC curves "are useful tools to select classification models" [115, p. 4]. They are created by plotting the TPR against the FPR while varying the classification thresholds of the underlying classifier. To characterize the performance of the classifier with a single metric, the area under the curve (AUC) of the ROC curve can be calculated according to $AUC(x) = \int_0^1 ROC(x)$. The diagonal of a ROC curve, along with its corresponding AUC value of $AUC(x) = 0.5$, can be interpreted as random guessing [115]. The ROC AUC value is the first of three metrics used to evaluate classification performance as part of this work.

The second method to assess the quality of the classifiers are precision-recall (PR) curves. PR curves result from plotting precision against recall while varying the classification thresholds of the underlying model. Similar to the ROC AUC, PR curves can be summarized with a single metric, the PR AUC. The PR AUC is defined as the area under the PR curve. In contrast to the ROC curve and its AUC value, the baseline value of a PR curve varies depending on the data set. Instead of a constant number (i.e. 0.5 for ROC AUC), the PR AUC baseline value is determined by $\frac{P}{P+N}$. It is the percentage of samples belonging to the positive class across all samples, making it especially suited for class-imbalanced data sets [114].

The third and final performance metric used as part of this work is the F_1 score, defined as the harmonic mean of precision and recall [116]:

$$F_1 = 2 * \frac{precision * recall}{precision + recall}$$

By taking into account both precision and recall, the F_1 score gives an overall indication of the performance of a classifier [116]. For this work, the best possible F_1 score of each classifier is used. It is calculated from the precision and recall values at the optimal decision threshold of the underlying model.

4.5.3 Total Number of Models

As stated at the beginning of Section 4.3: Evaluation, a total of 48 different permutations of the encoding are generated from the initial data sets. These result from all possible combinations of data set, recorded hierarchy level, and feature transformation strategy. These are combined with the four different dimensionality reduction methods, i.e. no dimensionality reduction, PCA, variable importance, and RFE as well as both random forest and SVM classifiers. This results in a total of $48 * 4 * 2 = 384$ trained machine learning models. The large number of training iterations makes computational optimizations such as parallel processing especially important.

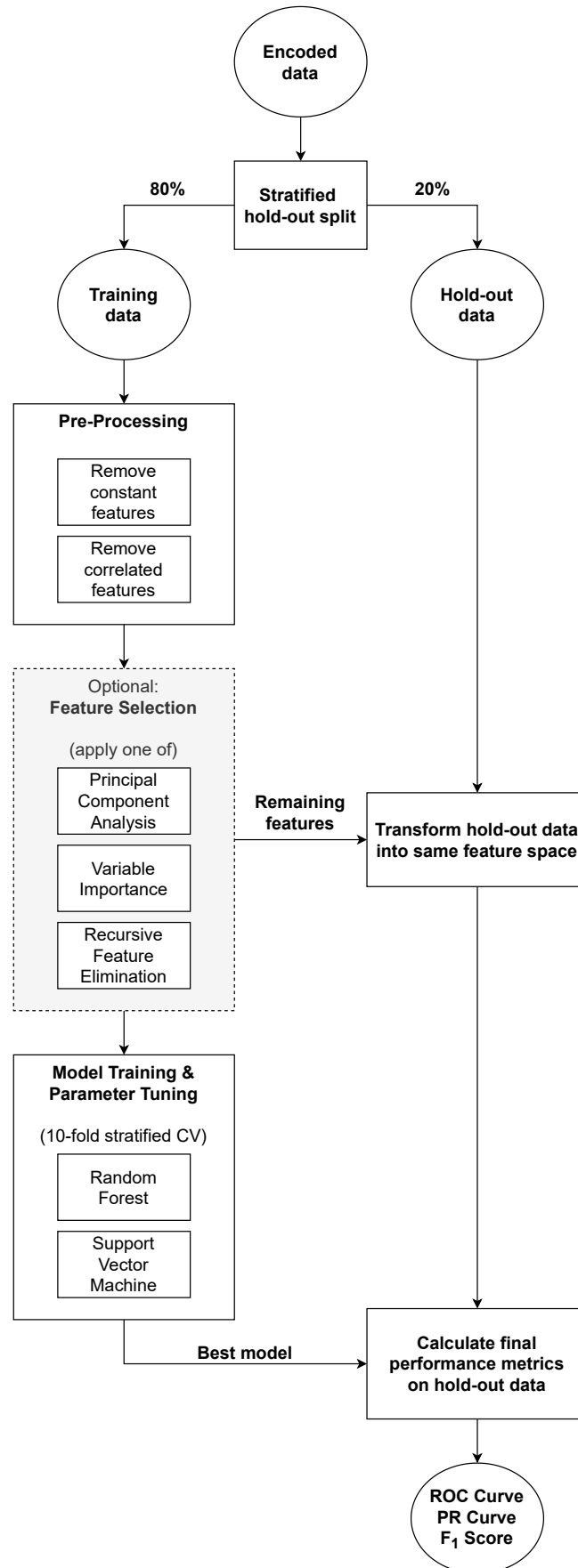


Figure 4.8: Machine learning pipeline to evaluate the proposed encoding.

5. Results

In the following sections, the number of retained features after feature selection as well as the classification results from the machine learning workflow are presented. Because of the large number of trained machine learning models, the specific focus is to answer the questions outlined in Section 1.2: Goals. Mainly, whether or not the proposed encoding is suitable to the task of encoding organic compounds; and which combination of parameter settings, dimensionality reduction technique and classification model can yield the best results.

Additionally, the results of the image generation step of the encoding are presented using the 20 proteinogenic amino acids as example compounds.

5.1 Dimensionality Reduction

With the pre-processing and feature selection steps, it was possible to reduce the average number of retained features across all encoded data sets by up to 90%. A full overview over the mean reduction in features for the pre-processing step and all three applied feature selection methods can be found in Table 5.1.

Method	Retained features
Pre-processing	63 %
Principal component analysis	5 %
Variable importance	26 %
Recursive feature elimination	10 %

Table 5.1: Average percentage of retained features after dimensionality reduction across all encoded data sets.

5.2 Classification Results

As stated in Section 4.5.3, a total number of 384 different classification models are trained. From the final performance metrics calculated with the hold-out subset, the following insights can be gained.

By comparing the F_1 scores of all trained classifiers using a t-test with a confidence level of 0.05, no statistically significant difference was found between the binary feature transformation strategy and the CPK-based discretization. Likewise, using the same confidence level of 0.05, no significant difference was found between the three available hierarchy levels. Additionally, none of the employed dimensionality reduction techniques resulted in significant differences of F_1 scores.

However, with a p-value of $p \approx 0.003$, the random forest model performed significantly better than the linear weighted support vector machine across all data sets. On average across all data sets, the best performance was achieved with the random forest model and RFE feature selection. In this case, only first level hierarchies were recorded and transformed using the binary transformation strategy. That model's complete performance metrics can be found in Table 5.2. The corresponding ROC and PR curves can be found in Figure 5.1 and Figure 5.2, respectively.

Data set	ROC AUC	F_1 Score
ace_vaxinpad	0.73	0.64
cpp_cellppd	0.82	0.77
cpp_cellppdmod	0.84	0.74
cpp_cppredfl	0.59	0.64
cpp_kelmcpp	0.71	0.63
cpp_mlcpp	0.73	0.62
cpp_mlcppue	0.61	0.68
hiv_protease	0.74	0.49

Table 5.2: Performance metrics for the best performing classification model overall. (Hierarchies: 1st level; Feature transformation: Binary; Classifier: Random forest; Feature selection: RFE)

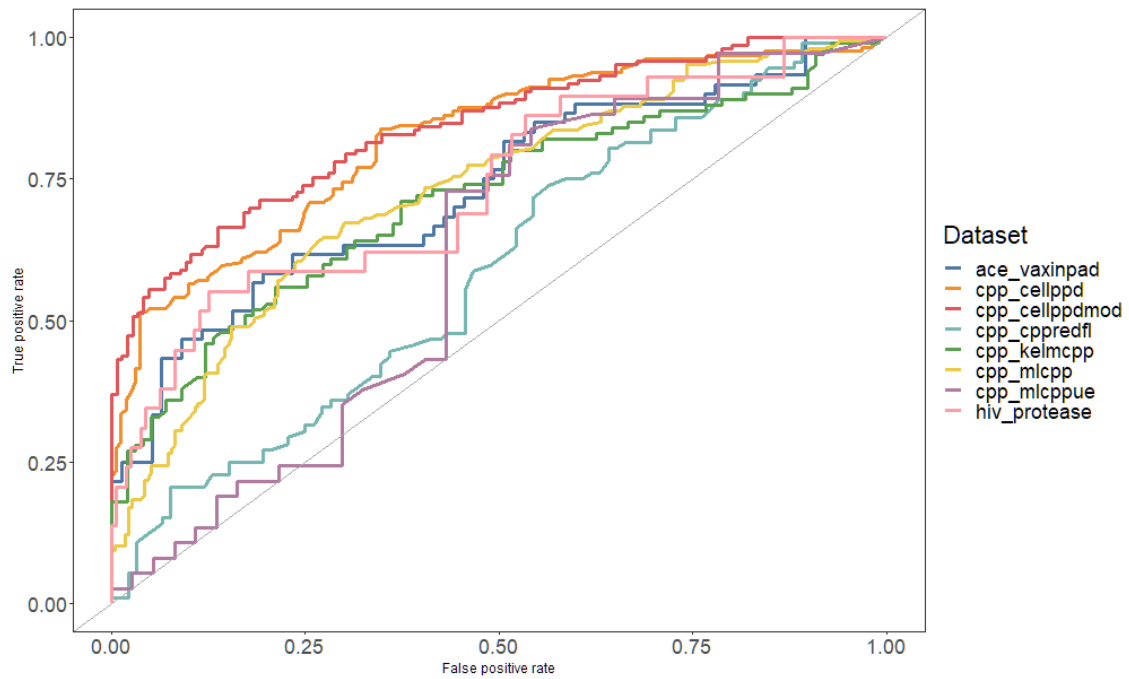


Figure 5.1: ROC curves for the best performing classification model overall.
(Hierarchies: 1st level; Feature transformation: Binary; Classifier: Random forest;
Feature selection: RFE)

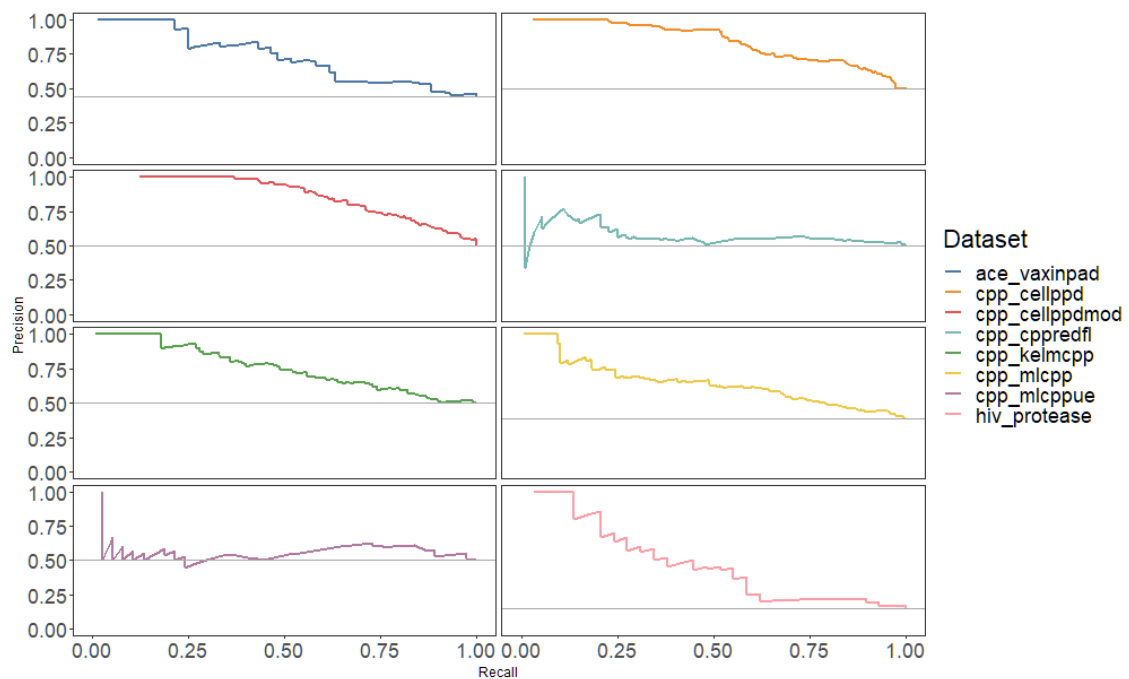


Figure 5.2: PR curves for the best performing classification model overall.
(Hierarchies: 1st level; Feature transformation: Binary; Classifier: Random forest;
Feature selection: RFE)

Best Results

In order to determine if certain parameter settings, dimensionality reduction methods, or machine learning models are better suited to a specific data set, Table 5.3 contains the best performing models per data set. The corresponding ROC and PR curves can be found in Figure 5.3 and Figure 5.4, respectively.

Data set	Encoding settings	Classifier settings	ROC AUC	F_1 Score
ace_vaxinpad	2 nd level; CPK	RF; All features	0.85	0.79
cpp_cellppd	1 st level; Binary	RF; RFE	0.82	0.77
cpp_cellppdmod	2 nd level; CPK	RF; RFE	0.89	0.83
cpp_cppredfl	1 st level; CPK	RF; RFE	0.69	0.72
cpp_kelmcpp	2 nd level; CPK	RF; PCA	0.73	0.75
cpp_mlcpp	1 st level; Binary	RF; All features	0.73	0.63
cpp_mlcppue	1 st level; Binary	SVM; PCA	0.64	0.71
hiv_protease	3 rd level; Binary	RF; All features	0.82	0.50

Table 5.3: Optimal classification model for each data set.

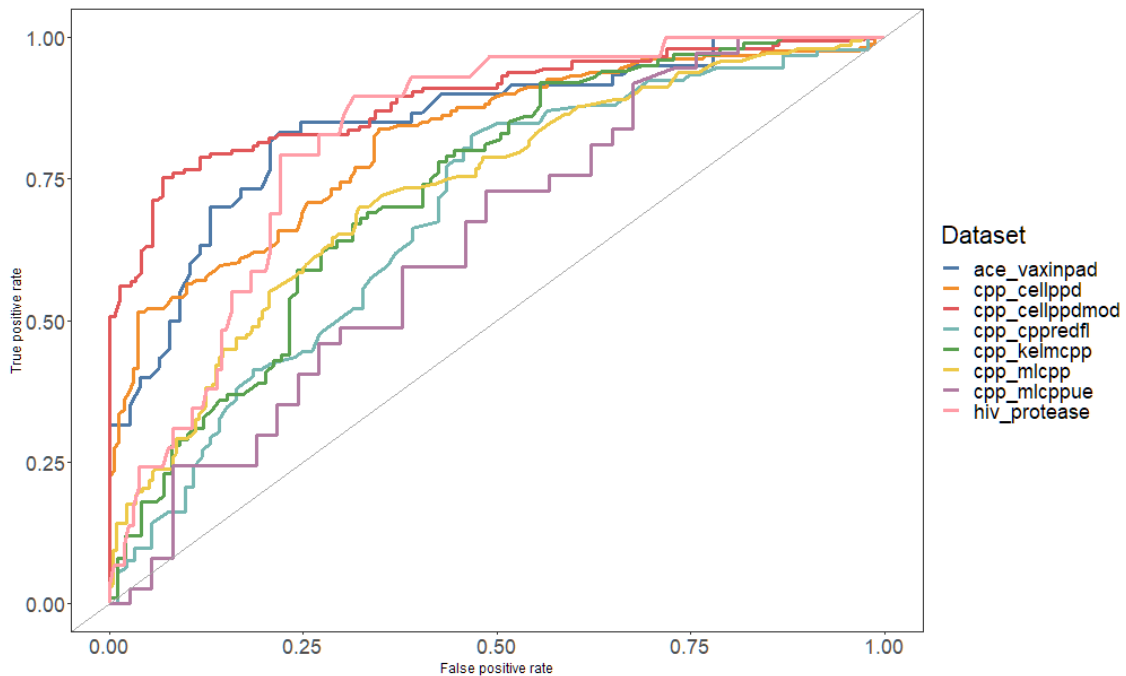


Figure 5.3: ROC curves for the best models per data set.

The corresponding encoding- and model parameters can be found in Table 5.3.

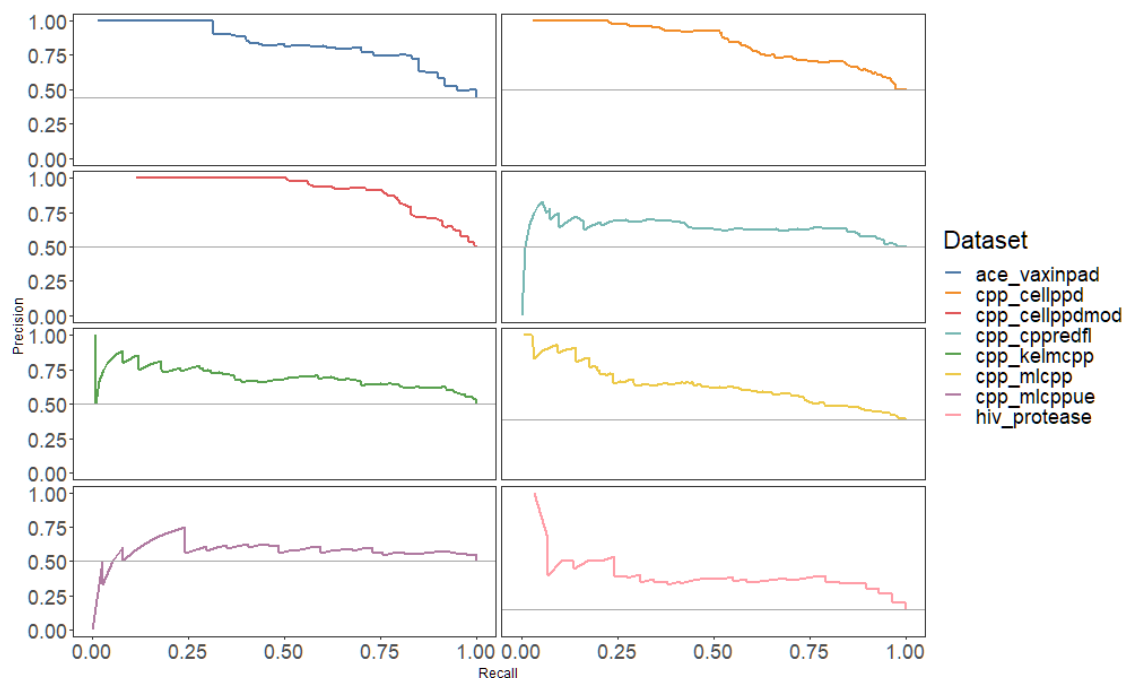


Figure 5.4: PR curves for the best models per data set.

The corresponding encoding- and model parameters can be found in Table 5.3.

The overall best result is achieved with the *cpp_cellppdmod* data set. A ROC AUC score of 0.89 and an F_1 score of 0.83 are reached when recording second level hierarchies and transforming them via CPK-based discretization. In this case, the random forest model is combined with RFE feature selection. Both the ROC and PR curves show that the model performs notably better than the respective baselines. Across the best results, the lowest AUC value of $AUC = 0.64$ was recorded for the *cpp_mlcppue* data set. The *cpp_mlcppue* is the smallest data set evaluated, containing only 374 observations.

5.3 Image Representations

To demonstrate the image representations that can be generated with the encoding, Figure 5.5 contains the images generated for the 20 proteinogenic amino acids. First level hierarchies were encoded and transformed using CPK-based discretization. From the image representations, the recorded neighborhoods along the carbon chain can be identified. For instance, for the cysteine amino acid, the oxygen atom (red), the nitrogen atom (blue), and the two sulfur atoms (yellow) can easily be seen. In addition, more subtle aspects like the minor structural difference between leucine and isoleucine are immediately apparent from the arrangement of carbon (black)

and hydrogen (white) blocks in the respective images. Even shared molecular features, like the aromatic rings present in phenylalanine, tryptophan and tyrosine, manifest as the same step-like carbon (black) structure.

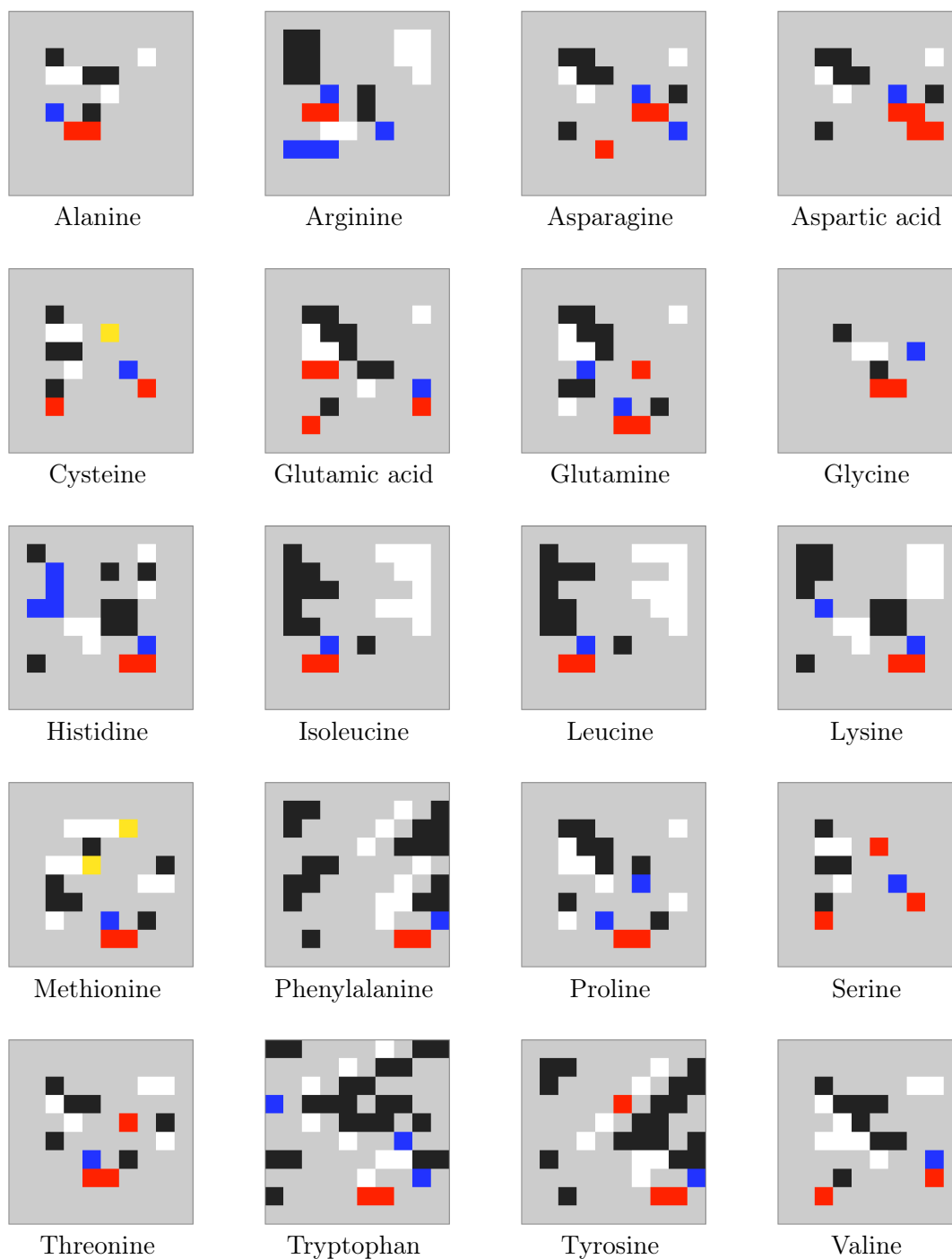


Figure 5.5: Image representations of the 20 proteinogenic amino acids.

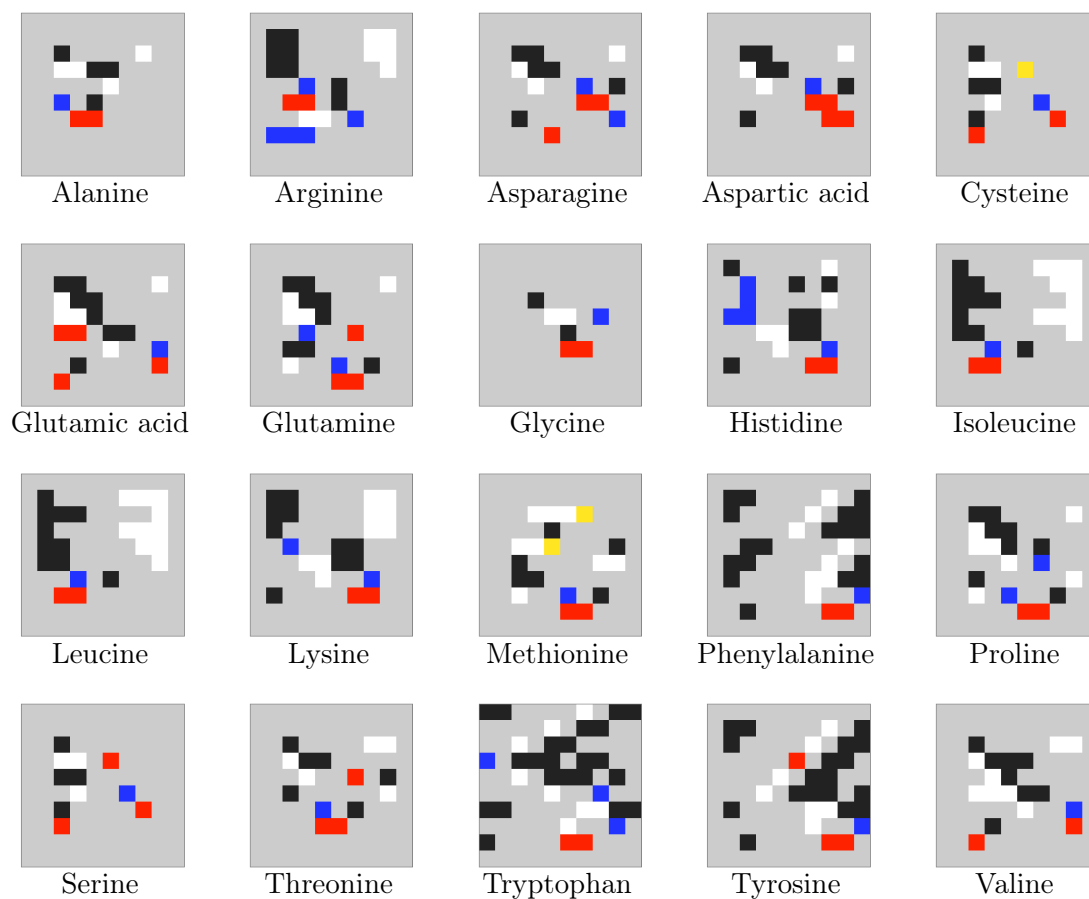


Figure 5.6: Image representations of the 20 proteinogenic amino acids.

6. Discussion

The proposed encoding offers very promising initial performance on the evaluated data sets. For all data sets, optimal ROC AUC values of well over > 0.5 were achieved, with the lowest values being $AUC = 0.64$ and the highest $AUC = 0.89$. This supports the idea of using a neighborhood-based encoding for molecules. The best performance metrics of $AUC > 0.8$ achieved for two of the peptide data sets, namely *cpp_cellppd* and *cpp_cellppdmod*, are as high as the results achieved by other research into CPP classification by Gautam *et. al* and Manavalan *et. al* [44, 45].

Parameters

The two implemented parameters, namely the number of encoded hierarchies and the feature transformation strategy, do not influence the performance significantly. This might be due to several factors. First, the evaluated data consists mainly of peptides. In particular, CPPs are considered small chains of amino acids [46]. Because of this, encoding additional hierarchies further away from the carbon chain might not provide any relevant information. Rather, the elements immediately surrounding the carbon chain characterize the compounds. Moreover, applying CPK-based discretization to the encoded data only amplifies information that is already implicitly present in the feature vectors. Since the transformation from categorical neighborhoods to binary or numeric features uses dummy encoding, the type of atom is implicitly contained in the additionally generated columns. Furthermore, it is likely that simply encoding the presence or absence of any atom in the respective neighborhoods already provides enough information for correct classification.

For four of the data sets, namely *ace_vaxinpad*, *cpp_cellppdmod*, *cpp_kelmcpp*, and *hiv_protease*, the best classification results were achieved by encoding more than one hierarchy level. However, no statistically significant difference in classification performance was found between the different hierarchy levels across all data sets. Therefore, the observation made for the four aforementioned data sets cannot be generalized. But, the observation indicates that recording additional hierarchy levels can result in a better molecular representation for some compounds.

Data Structure

Generally, the class-balanced data sets performed better in the classification task. This observation is consistent across other machine learning applications [117]. But even without the application of any oversampling techniques, the highly class-imbalanced *hiv_protease* data set achieved good classification results of $AUC = 0.82$ and $F_1 = 0.5$. Additionally, from the PR curves found in Figure 5.4, it can be seen that the model performs better than the baseline for all classification thresholds. This further supports the idea of using a neighborhood-based encoding for molecules, even for more difficult data sets. The inclusion of data sets with varying numbers of observations established the usefulness of the encoding for different data set sizes.

Feature Selection

None of the employed feature selection approaches displayed any significant effect on classification accuracy. Both random forest and support vector machine classifiers are known to be well suited when working with very high-dimensional data, potentially making any prior dimensionality reduction steps redundant. Random forests internally assign variable importance to the available features as part of the training process [108]. Likewise, SVM models implement a form of regularization with their cost parameter [118]. However, it was possible to reduce the computational time for training the classifiers by applying dimensionality reduction prior to training. This, in combination with the fact that there is no significant decrease in classification accuracy when including dimensionality reduction, can result in faster overall computational performance in future applications.

Model Choice

Overall, with a p-value of $p \approx 0.003$, the random forest model performed significantly better than the SVM. In contrast to the linear weighted support vector machine, the random forest model is able to classify data points non-linearly. Therefore, it is likely that for the evaluated data sets, distinguishing between the classes is less accurate with a linear decision boundary.

Furthermore, the inclusion of class weights in the SVM model did not result in better classification performance for the class-imbalanced data sets. Across the best achieved results, the SVM classifier only outperformed the random forest in one single case, namely for the *cyp_mlcypue* data set. However, with an imbalance ratio of $IR = 1$, the *cyp_mlcypue* is class-balanced.

Performance

By relying on optimized data structures, namely matrices and graphs, efficient computation is achieved during the encoding process. This is supplemented by parallel processing in the evaluation pipeline.

Additionally, by reducing the dimensionality of the encoded data sets using the employed feature selection methods, computational time can be further reduced [119].

Image Representations

The generated image representations offer a visual overview over the encoded compounds. By coloring the images using the CPK color scheme, the images are intuitive even without specific domain knowledge about computer science or molecular encodings. This increases the accessibility of the proposed encoding for use in biological and chemical research. The structure of the encoded neighborhoods as well as their elemental composition are immediately obvious from looking at the images.

Additional Applications

The similarity measures introduced in Chapter 2: Related Works, specifically the commonly used Tanimoto coefficient, can be applied directly to the generated binary feature vectors. Therefore, the proposed encoding can be used to solve the same tasks as other molecular fingerprints. Specifically, similarity searches, database querying, and virtual screening can be performed with compounds encoded by the proposed encoding.

7. Conclusion

It was examined whether the neighborhoods around the carbon chain of organic compounds can be used to generate a suitable molecular representation. For this, a novel neighborhood-based encoding was developed and subsequently applied to eight different data sets. The encoded data was evaluated using machine learning in a binary classification task to predict biological properties. Very promising results were achieved in the initial evaluation. With maximum AUC values of $AUC > 0.8$, the achieved performance metrics were on par with similar studies. Therefore, the results support the idea of encoding atomic neighborhoods around the carbon chain to represent organic molecules. Due to the limited number of evaluated data sets, additional evaluation on different tasks has to be performed in the future. This is made simple by multiple factors. First, no significant changes in classification accuracy were found between the different parameter settings made available by the encoding. For future evaluation, these parameters do not have to be considered and evaluation can solely focus on different tasks using more data sets. Secondly, no significant differences were found between the employed dimensionality reduction techniques. Therefore, future evaluation can be performed using the dimensionality reduction technique that offers the fastest computational performance by eliminating the maximum number of features. Lastly, since special care was taken in implementing the encoding and evaluation pipeline in a user-friendly manner by providing optional parameters for all configurable steps, no modifications have to be made to assess additional data sets.

The modular nature of the implementation in combination with the flexibility of the SMILES format also enables the extension of the encoding algorithm itself. Presently, only the existence of bonds is taken into account when determining neighboring elements. This could be extended to include other chemical properties like the specific type of bond.

Moreover, the generated image representations can be used to solve advanced tasks. For instance, temporal changes within compounds could be investigated by gener-

ating images at several points in time and identifying the visual differences. Alternatively, image-based deep learning algorithms like convolutional neural networks can be trained on the images to enable state-of-the-art analyses.

Glossary

ADMET Abbreviation for "absorption, distribution, metabolism, excretion, toxicity"
- describing the disposition of a pharmaceutical compound in an organism.

Antigen Presenting Cells A group of immune cells that mediate the cellular immune response.

Cytochrome Proteins that serve a vital function in the transfer of energy within cells.

Enzyme Proteins that act as catalysts, accelerating chemical reactions.

Hydrocarbon An organic compound consisting entirely of hydrogen and carbon.

Hyperparameter A parameter to control the training process.

Nucleotide An organic molecule and the basic building block of nucleic acids.

Peptide Short chains of two to fifty amino acids linked by peptide bonds.

Phenol An aromatic organic compound with the molecular formula C_6H_5OH or C_6H_6O .

Protease An enzyme that catalyzes the breakdown of proteins.

Proteinogenic Protein creating.

Valence The measure of an element's combining power with other atoms.

Acronyms

API application programming interface.

AUC area under the curve.

CPK Corey–Pauling–Koltun.

CPP cell-penetrating peptide.

CPU central processing unit.

CV cross-validation.

DBN deep belief network.

DNN deep neural network.

DTI drug-target interaction.

ECFP Extended-Connectivity Fingerprint.

FASTA FAST-All.

FN false negative.

FP false positive.

FPR false positive rate.

HIV human immunodeficiency virus.

IR imbalance ratio.

MACCS Molecular Access System.

MDS multidimensional scaling.

MNA Multilevel Neighborhoods of Atoms.

PCA principal component analysis.

PR precision-recall.

QSAR quantitative structure-activity relationship.

RF random forest.

RFE recursive feature elimination.

ROC receiver operating characteristic.

SMILES simplified molecular-input line-entry system.

SVM support vector machine.

t-SNE t-distributed stochastic neighbor embedding.

TN true negative.

TP true positive.

TPR true positive rate.

References

- [1] Stephani Joy Y Macalino et al. “Role of computer-aided drug design in modern drug discovery”. In: *Archives of pharmacal research* 38.9 (2015), pp. 1686–1701.
- [2] Wen Zhang et al. “Recent advances in the machine learning-based drug-target interaction prediction”. In: *Current drug metabolism* 20.3 (2019), pp. 194–202.
- [3] Kyaw-Zeyar Myint et al. “Molecular fingerprint-based artificial neural networks QSAR for ligand biological activity predictions”. In: *Molecular pharmacology* 9.10 (2012), pp. 2912–2923.
- [4] Sebastian Spänig and Dominik Heider. “Encodings and models for antimicrobial peptide classification for multi-resistant pathogens”. In: *BioData Mining* 12.1 (2019), p. 7.
- [5] Dávid Bajusz, Anita Rácz, and Károly Héberger. “Chemical data formats, fingerprints, and other molecular descriptions for database analysis and searching”. In: *In Silico Drug Discovery Tools*. Elsevier Inc., 2017, pp. 329–378.
- [6] Andreas Hirsch. “The era of carbon allotropes”. In: *Nature materials* 9.11 (2010), pp. 868–871.
- [7] Adrià Cereto-Massagué et al. “Molecular fingerprint similarity search in virtual screening”. In: *Methods* 71 (2015), pp. 58–63.
- [8] Daniel A Keim et al. “Challenges in visual data analysis”. In: *Tenth International Conference on Information Visualisation (IV’06)*. IEEE. 2006, pp. 9–16.
- [9] Sereina Riniker and Gregory A Landrum. “Similarity maps-a visualization strategy for molecular fingerprints and machine-learning methods”. In: *Journal of cheminformatics* 5.1 (2013), p. 43.

- [10] Kathrin Heikamp and Jürgen Bajorath. “How do 2D fingerprints detect structurally diverse active compounds? Revealing compound subset-specific fingerprint features through systematic selection”. In: *Journal of chemical information and modeling* 51.9 (2011), pp. 2254–2265.
- [11] Dávid Bajusz, Anita Rácz, and Károly Héberger. “Why is Tanimoto index an appropriate choice for fingerprint-based similarity calculations?” In: *Journal of cheminformatics* 7.1 (2015), p. 20.
- [12] Joseph L Durant et al. “Reoptimization of MDL keys for use in drug discovery”. In: *Journal of chemical information and computer sciences* 42.6 (2002), pp. 1273–1280.
- [13] Evan E Bolton et al. “PubChem: integrated platform of small molecules and biological activities”. In: *Annual reports in computational chemistry*. Vol. 4. Elsevier, 2008, pp. 217–241.
- [14] Zhuang Wang et al. “In silico prediction of blood–brain barrier permeability of compounds by machine learning and resampling methods”. In: *ChemMedChem* 13.20 (2018), pp. 2189–2201.
- [15] Noel M O’Boyle et al. “Open Babel: An open chemical toolbox”. In: *Journal of cheminformatics* 3.1 (2011), p. 33.
- [16] Robert C Glen et al. “Circular fingerprints: flexible molecular descriptors with applications from physical chemistry to ADME”. In: *IDrugs* 9.3 (2006), p. 199.
- [17] David Rogers and Mathew Hahn. “Extended-connectivity fingerprints”. In: *Journal of chemical information and modeling* 50.5 (2010), pp. 742–754.
- [18] Berith F Jensen et al. “In silico prediction of cytochrome P450 2D6 and 3A4 inhibition using Gaussian kernel weighted k-nearest neighbor and extended connectivity fingerprints, including structural fragment analysis of inhibitors versus noninhibitors”. In: *Journal of medicinal chemistry* 50.3 (2007), pp. 501–511.
- [19] Dmitrii Filimonov et al. “Chemical similarity assessment through multilevel neighborhoods of atoms: definition and comparison with the other descriptors”. In: *Journal of chemical information and computer sciences* 39.4 (1999), pp. 666–670.
- [20] Certara, Inc. URL: <https://www.certara.com/> (visited on 12/17/2020).

- [21] CG Wermuth et al. “Glossary of terms used in medicinal chemistry (IUPAC Recommendations 1998)”. In: *Pure and Applied Chemistry* 70.5 (1998), pp. 1129–1143.
- [22] Xiaoyu Qing et al. “Pharmacophore modeling: advances, limitations, and current utility in drug discovery”. In: *Journal of Receptor, Ligand and Channel Research* 7 (2014), pp. 81–92.
- [23] Mark A Johnson and Gerald M Maggiora. *Concepts and applications of molecular similarity*. Wiley, 1990.
- [24] Peter Willett. “Similarity-based virtual screening using 2D fingerprints”. In: *Drug discovery today* 11.23-24 (2006), pp. 1046–1053.
- [25] Jérôme Hert et al. “Comparison of fingerprint-based methods for virtual screening using multiple bioactive reference structures”. In: *Journal of chemical information and computer sciences* 44.3 (2004), pp. 1177–1185.
- [26] Sunyoung Kwon et al. “Comprehensive ensemble in QSAR prediction for drug discovery”. In: *BMC bioinformatics* 20.1 (2019), p. 521.
- [27] Bruno J Neves et al. “QSAR-based virtual screening: advances and applications in drug discovery”. In: *Frontiers in pharmacology* 9 (2018), p. 1275.
- [28] Ignacio Ponzoni et al. “QSAR Classification Models for Predicting the Activity of Inhibitors of Beta-Secretase (BACE1) Associated with Alzheimer’s Disease”. In: *Scientific reports* 9.1 (2019), pp. 1–13.
- [29] Jaykant Vora et al. “Molecular docking, QSAR and ADMET based mining of natural compounds against prime targets of HIV”. In: *Journal of Biomolecular Structure and Dynamics* 37.1 (2019), pp. 131–146.
- [30] HÅvard Jenssen et al. “QSAR modeling and computer-aided design of antimicrobial peptides”. In: *Journal of peptide science: an official publication of the European Peptide Society* 14.1 (2008), pp. 110–114.
- [31] P Deepak and Prasad M Deshpande. *Operators for similarity search: Semantics, techniques and usage scenarios*. Springer, 2015.
- [32] Anita RÁCz, DÁvid Bajusz, and KÁroly Héberger. “Life beyond the Tanimoto coefficient: similarity measures for interaction fingerprints”. In: *Journal of Cheminformatics* 10.1 (2018), p. 48.

- [33] Jeffrey W Godden, Ling Xue, and Jürgen Bajorath. “Combinatorial preferences affect molecular similarity/diversity calculations using binary fingerprints and Tanimoto coefficients”. In: *Journal of Chemical Information and Computer Sciences* 40.1 (2000), pp. 163–166.
- [34] Maged Nasser et al. “Deep Belief Network for Molecular Feature Selection in Ligand-Based Virtual Screening”. In: *International Conference of Reliable Information and Communication Technology*. Springer. 2018, pp. 3–14.
- [35] Andreas Bender et al. “Molecular similarity searching using atom environments, information-based feature selection, and a naive Bayesian classifier”. In: *Journal of chemical information and computer sciences* 44.1 (2004), pp. 170–178.
- [36] Abdelrahman Saad, Fahima A Maghraby, and Yasser M Omar. “Predicting drug target interaction by integrating drug fingerprint and drug side effect using machine learning”. In: *International Conference on Advanced Machine Learning Technologies and Applications*. Springer. 2019, pp. 281–290.
- [37] Sofia D’Souza, KV Prema, and Seetharaman Balaji. “Machine learning models for drug–target interactions: current knowledge and future directions”. In: *Drug Discovery Today* 25.4 (2020), pp. 748–756.
- [38] Loris Nanni, Alessandra Lumini, and Sheryl Brahnam. “An empirical study of different approaches for protein classification”. In: *The Scientific World Journal* 2014 (2014).
- [39] Taeho Jo and Jianlin Cheng. “Improving protein fold recognition by random forest”. In: *BMC bioinformatics*. Vol. 15. S11. Springer. 2014, S14.
- [40] CZ Cai et al. “Protein function classification via support vector machine approach”. In: *Mathematical biosciences* 185.2 (2003), pp. 111–122.
- [41] Seonwoo Min, Byunghan Lee, and Sungroh Yoon. “Deep learning in bioinformatics”. In: *Briefings in bioinformatics* 18.5 (2017), pp. 851–869.
- [42] Justin M Wolfe et al. “Machine learning to predict cell-penetrating peptides for antisense delivery”. In: *ACS central science* 4.4 (2018), pp. 512–520.
- [43] William S Sanders et al. “Prediction of cell penetrating peptides by support vector machines”. In: *PLoS Comput Biol* 7.7 (2011), e1002101.
- [44] Ankur Gautam et al. “In silico approaches for designing highly effective cell penetrating peptides”. In: *Journal of translational medicine* 11.1 (2013), p. 74.

- [45] Balachandran Manavalan et al. “Machine-learning-based prediction of cell-penetrating peptides and their uptake efficiency with improved accuracy”. In: *Journal of proteome research* 17.8 (2018), pp. 2715–2726.
- [46] Frederic Heitz, May Catherine Morris, and Gilles Divita. “Twenty years of cell-penetrating peptides: from molecular mechanisms to therapeutics”. In: *British journal of pharmacology* 157.2 (2009), pp. 195–206.
- [47] Maria Lindgren et al. “Cell-penetrating peptides”. In: *Trends in pharmacological sciences* 21.3 (2000), pp. 99–103.
- [48] Amalia Luque et al. “The impact of class imbalance in classification performance metrics based on the binary confusion matrix”. In: *Pattern Recognition* 91 (2019), pp. 216–231.
- [49] Jamey D Marth. “A unified vision of the building blocks of life”. In: *Nature cell biology* 10.9 (2008), pp. 1015–1015.
- [50] Sunghwan Kim et al. “PubChem in 2021: new data content and improved web interfaces”. In: *Nucleic Acids Research* 49.D1 (2021), pp. D1388–D1395.
- [51] Gandharva Nagpal et al. “Computer-aided prediction of antigen presenting cell modulators for designing peptide-based vaccine adjuvants”. In: *Journal of translational medicine* 16.1 (2018), p. 181.
- [52] Vinod Kumar et al. “Prediction of cell-penetrating potential of modified peptides containing natural and chemically modified residues”. In: *Frontiers in microbiology* 9 (2018), p. 725.
- [53] Leyi Wei, Jijun Tang, and Quan Zou. “SkipCPP-Pred: an improved and promising sequence-based predictor for predicting cell-penetrating peptides”. In: *BMC genomics* 18.7 (2017), pp. 1–11.
- [54] Poonam Pandey et al. “KELM-CPPpred: Kernel extreme learning machine based prediction model for cell-penetrating peptides”. In: *Journal of proteome research* 17.9 (2018), pp. 3214–3222.
- [55] Thorsteinn Rögnvaldsson, Liwen You, and Daniel Garwicz. “State of the art prediction of HIV-1 protease cleavage sites”. In: *Bioinformatics* 31.8 (2015), pp. 1204–1210.
- [56] David Weininger. “SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules”. In: *Journal of chemical information and computer sciences* 28.1 (1988), pp. 31–36.

- [57] Guido Van Rossum and Fred L. Drake. *Python 3 Reference Manual*. Scotts Valley, CA: CreateSpace, 2009. ISBN: 1441412697.
- [58] Travis E Oliphant. “Python for scientific computing”. In: *Computing in Science & Engineering* 9.3 (2007), pp. 10–20.
- [59] Peter JA Cock et al. “Biopython: freely available Python tools for computational molecular biology and bioinformatics”. In: *Bioinformatics* 25.11 (2009), pp. 1422–1423.
- [60] J. D. Hunter. “Matplotlib: A 2D graphics environment”. In: *Computing in Science & Engineering* 9.3 (2007), pp. 90–95. DOI: 10.1109/MCSE.2007.55.
- [61] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. “Exploring Network Structure, Dynamics, and Function using NetworkX”. In: *Proceedings of the 7th Python in Science Conference*. Ed. by Gaël Varoquaux, Travis Vaught, and Jarrod Millman. Pasadena, CA USA, 2008, pp. 11–15.
- [62] Charles R. Harris et al. “Array programming with NumPy”. In: *Nature* 585 (2020), pp. 357–362. DOI: 10.1038/s41586-020-2649-2.
- [63] The pandas development team. *pandas-dev/pandas: Pandas*. Version latest. Feb. 2020. DOI: 10.5281/zenodo.3509134. URL: <https://doi.org/10.5281/zenodo.3509134>.
- [64] RDKit. *RDKit: Open-source cheminformatics*. <http://www.rdkit.org>. (Visited on 11/04/2020).
- [65] Peter C Kroon. *pysmiles*. <https://github.com/pckroon/pysmiles>. (Visited on 11/10/2020).
- [66] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. Vienna, Austria, 2020. URL: <https://www.R-project.org/>.
- [67] Max Kuhn. *caret: Classification and Regression Training*. R package version 6.0-86. 2020. URL: <https://CRAN.R-project.org/package=caret>.
- [68] Microsoft Corporation and Steve Weston. *doParallel: Foreach Parallel Adaptor for the 'parallel' Package*. R package version 1.0.16. 2020. URL: <https://CRAN.R-project.org/package=doParallel>.
- [69] Hadley Wickham et al. *dplyr: A Grammar of Data Manipulation*. R package version 1.0.2. 2020. URL: <https://CRAN.R-project.org/package=dplyr>.

- [70] David Meyer et al. *e1071: Misc Functions of the Department of Statistics, Probability Theory Group (Formerly: E1071), TU Wien*. R package version 1.7-4. 2020. URL: <https://CRAN.R-project.org/package=e1071>.
- [71] Hadley Wickham. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York, 2016. ISBN: 978-3-319-24277-4. URL: <https://ggplot2.tidyverse.org>.
- [72] Alboukadel Kassambara. *ggpubr: 'ggplot2' Based Publication Ready Plots*. R package version 0.4.0. 2020. URL: <https://CRAN.R-project.org/package=ggpubr>.
- [73] Jeffrey B. Arnold. *ggthemes: Extra Themes, Scales and Geoms for 'ggplot2'*. R package version 4.2.0. 2019. URL: <https://CRAN.R-project.org/package=ggthemes>.
- [74] Sam Firke. *janitor: Simple Tools for Examining and Cleaning Dirty Data*. R package version 2.0.1. 2020. URL: <https://CRAN.R-project.org/package=janitor>.
- [75] Christopher R John. *MLeval: Machine Learning Model Evaluation*. R package version 0.3. 2020. URL: <https://CRAN.R-project.org/package=MLeval>.
- [76] Bernd Bischl et al. “mlr: Machine Learning in R”. In: *Journal of Machine Learning Research* 17.170 (2016), pp. 1–5. URL: <https://jmlr.org/papers/v17/15-066.html>.
- [77] Marvin N. Wright and Andreas Ziegler. “ranger: A Fast Implementation of Random Forests for High Dimensional Data in C++ and R”. In: *Journal of Statistical Software* 77.1 (2017), pp. 1–17. DOI: 10.18637/jss.v077.i01.
- [78] Manos Papadakis et al. *Rfast: A Collection of Efficient and Extremely Fast R Functions*. R package version 2.0.1. 2020. URL: <https://CRAN.R-project.org/package=Rfast>.
- [79] Joerg Kurt Wegner et al. “Cheminformatics”. In: *Communications of the ACM* 55.11 (2012), pp. 65–75.
- [80] Clemens Arbesser et al. “Visplause: Visual data quality assessment of many time series using plausibility checks”. In: *IEEE transactions on visualization and computer graphics* 23.1 (2016), pp. 641–650.
- [81] Paolo Panaroni et al. “Safety in automotive software: An overview of current practices”. In: *2008 32nd Annual IEEE International Computer Software and Applications Conference*. IEEE. 2008, pp. 1053–1058.

- [82] Wikipedia contributors. *Phenol — Wikipedia, The Free Encyclopedia*. 2021. URL: <https://en.wikipedia.org/w/index.php?title=Phenol&oldid=997847649> (visited on 02/04/2021).
- [83] Lukas Meier, Sara Van De Geer, and Peter Bühlmann. “The group lasso for logistic regression”. In: *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 70.1 (2008), pp. 53–71.
- [84] Rajni Bhalla and Amandeep Bagga. “Opinion mining framework using proposed rb-bayes model for text classification.” In: *International Journal of Electrical & Computer Engineering (2088-8708)* 9.1 (2019).
- [85] Susan Garavaglia and Asha Sharma. “A smart guide to dummy variables: Four applications and a macro”. In: *Proceedings of the northeast SAS users group conference*. Vol. 43. 1998.
- [86] Robert B Corey and Linus Pauling. “Molecular models of amino acids, peptides, and proteins”. In: *Review of Scientific Instruments* 24.8 (1953), pp. 621–627.
- [87] Walter L Koltun. *Space filling atomic units and connectors for molecular models*. US Patent 3,170,246. Feb. 1965.
- [88] Robert M Esnouf. “An extensively modified version of MolScript that includes greatly enhanced coloring capabilities”. In: *Journal of Molecular Graphics and Modelling* 15.2 (1997), pp. 132–134.
- [89] John L Moreland et al. “The Molecular Biology Toolkit (MBT): a modular platform for developing molecular visualization applications”. In: *BMC bioinformatics* 6.1 (2005), pp. 1–7.
- [90] Jianping Hua et al. “Optimal number of features as a function of sample size for various classification rules”. In: *Bioinformatics* 21.8 (2005), pp. 1509–1515.
- [91] S Velliangiri, S Alagumuthukrishnan, et al. “A review of dimensionality reduction techniques for efficient computation”. In: *Procedia Computer Science* 165 (2019), pp. 104–111.
- [92] Jundong Li et al. “Feature selection: A data perspective”. In: *ACM Computing Surveys (CSUR)* 50.6 (2017), pp. 1–45.
- [93] Lei Yu and Huan Liu. “Feature selection for high-dimensional data: A fast correlation-based filter solution”. In: *Proceedings of the 20th international conference on machine learning (ICML-03)*. 2003, pp. 856–863.

- [94] Mark Andrew Hall. “Correlation-based feature selection for machine learning”. In: (1999).
- [95] Bruce Ratner. “The correlation coefficient: Its values range between $+1/-1$, or do they?” In: *Journal of targeting, measurement and analysis for marketing* 17.2 (2009), pp. 139–142.
- [96] Suresh Venkatasubramanian and Qiushi Wang. “The Johnson-Lindenstrauss transform: an empirical study”. In: *2011 Proceedings of the Thirteenth Workshop on Algorithm Engineering and Experiments (ALENEX)*. SIAM. 2011, pp. 164–173.
- [97] Ella Bingham and Heikki Mannila. “Random projection in dimensionality reduction: applications to image and text data”. In: *Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining*. 2001, pp. 245–250.
- [98] Svante Wold, Kim Esbensen, and Paul Geladi. “Principal component analysis”. In: *Chemometrics and intelligent laboratory systems* 2.1-3 (1987), pp. 37–52.
- [99] Michael AA Cox and Trevor F Cox. “Multidimensional scaling”. In: *Handbook of data visualization*. Springer, 2008, pp. 315–347.
- [100] Laurens Van der Maaten and Geoffrey Hinton. “Visualizing data using t-SNE.” In: *Journal of machine learning research* 9.11 (2008).
- [101] Jelena Zubova, Olga Kurasova, and Marius Liutvinavičius. “Dimensionality reduction methods: The comparison of speed and accuracy”. In: *Information Technology and Control* 47.1 (2018), pp. 151–160.
- [102] Mu Zhu and Ali Ghodsi. “Automatic dimensionality selection from the scree plot via the use of profile likelihood”. In: *Computational Statistics & Data Analysis* 51.2 (2006), pp. 918–930.
- [103] William R Zwick and Wayne F Velicer. “Comparison of five rules for determining the number of components to retain.” In: *Psychological bulletin* 99.3 (1986), p. 432.
- [104] Alethea Rea and William Rea. “How many components should be retained from a multivariate time series PCA?” In: *arXiv preprint arXiv:1610.03588* (2016).
- [105] Roman Timofeev. “Classification and regression trees (CART) theory and applications”. In: *Humboldt University, Berlin* (2004), pp. 1–40.

- [106] Leszek Rutkowski et al. “The CART decision tree for mining data streams”. In: *Information Sciences* 266 (2014), pp. 1–15.
- [107] Purnima Bholowalia and Arvind Kumar. “EBK-means: A clustering technique based on elbow method and k-means in WSN”. In: *International Journal of Computer Applications* 105.9 (2014).
- [108] Leo Breiman. “Random forests”. In: *Machine learning* 45.1 (2001), pp. 5–32.
- [109] Marti A. Hearst et al. “Support vector machines”. In: *IEEE Intelligent Systems and their applications* 13.4 (1998), pp. 18–28.
- [110] Ashish Anand et al. “An approach for classification of highly imbalanced data using weighting and undersampling”. In: *Amino acids* 39.5 (2010), pp. 1385–1391.
- [111] Andrew W Moore. “Cross-validation for detecting and preventing overfitting”. In: *School of Computer Science Carnegie Mellon University* (2001).
- [112] Andrius Vabalas et al. “Machine learning algorithm validation with a limited sample size”. In: *PloS one* 14.11 (2019), e0224365.
- [113] Jerzy Neyman. “On the two different aspects of the representative method: the method of stratified sampling and the method of purposive selection”. In: *Breakthroughs in Statistics*. Springer, 1992, pp. 123–150.
- [114] Jesse Davis and Mark Goadrich. “The relationship between Precision-Recall and ROC curves”. In: *Proceedings of the 23rd international conference on Machine learning*. 2006, pp. 233–240.
- [115] Sebastian Raschka. “An overview of general performance metrics of binary classifier systems”. In: *arXiv preprint arXiv:1410.5330* (2014).
- [116] Guy S Handelman et al. “Peering into the black box of artificial intelligence: evaluation metrics of machine learning methods”. In: *American Journal of Roentgenology* 212.1 (2019), pp. 38–43.
- [117] Xinjian Guo et al. “On the class imbalance problem”. In: *2008 Fourth international conference on natural computation*. Vol. 4. IEEE. 2008, pp. 192–201.
- [118] Trevor Hastie et al. “The entire regularization path for the support vector machine”. In: *Journal of Machine Learning Research* 5.Oct (2004), pp. 1391–1415.

- [119] F. Pedregosa et al. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830. URL: https://scikit-learn.org/0.15/modules/computational_performance.html#influence-of-the-number-of-features (visited on 03/11/2021).